

PropertyKindCalculus Blueprint

Compiled

2026-06-15T00:24:47Z

Project `86a8bd7`

staged docs: re-render blueprint with the full ISO 80000-3 catalogue

Lean

leanprover/lean4:v4.30.0

VersoBlueprint (<https://github.com/leanprover/verso-blueprint>)

`v4.30.0@01a3640805c9` (<https://github.com/leanprover/verso-blueprint/commit/01a3640805c9bb11528972c4d1b78590489f826e>)

Mathlib (<https://github.com/leanprover-community/mathlib4>)

`v4.30.0@c5ea00351c28` (<https://github.com/leanprover-community/mathlib4/commit/c5ea00351c28e24afc9f0f84379aa41082b1188f>)

PropertyKindCalculus is a Lean 4 formalization of René Dybkær's **An Ontology on Property for Physical, Chemical, and Biological Systems** (2009), extended with David Flater's full tracking of kinds of quantities (NIST Technical Note 1943, Appendix C). This blueprint is the design map: it records what is already proved (linked to real declarations) and the **capstone theorems we plan to provide**, with the dependency graph and a status summary at the end.

The whole blueprint is also available as a single paginated document: [download the PDF](#).

Table of Contents

1. Requirements
2. Why a calculus, not a taxonomy
3. How to read the status
4. The Proved Spine
5. Units and the Dimension-1 Problem
6. Dimension as a Forgetful Functor
7. The Interaction Algebra (Flater Appendix C)
8. Extensivity
9. ISO/IEC 80000 — The Standards Catalogue
10. ISO 80000-3 — Space and Time

1. Requirements

What follows is the fixed set of requirements PropertyKindCalculus is **specified** to meet. The next section, **Why a calculus, not a taxonomy**, argues **why** a calculus is needed where a taxonomy stops, and the chapters after it develop and prove each requirement; this section states them up front, as the axes the design is judged against, with a status table at the end mapping each axis to the declaration that discharges it.

The requirements fall into four groups: how kinds are **structured**, how **operations** on them are gated, how the kind layer is kept **consistent** with the coarser dimension and unit layers, and how values **aggregate** over parts.

1.1. Kind structure

R1 — Kinds are first-class and discriminate within a dimension. Two properties of the same physical dimension can still be different kinds, and the type system must keep them apart. Volumetric water content (Quantity vwc), gravimetric water content (Quantity gwc), relative permittivity, reflectivity, and emissivity are all dimension-one, yet Quantity vwc and Quantity gwc must be **different types** — not interchangeable values. (QUDT's and SysML v2's QuantityKind is subsumed here as one such kind — a scale-gated leaf — not as the root.)

R2 — Specialization is a lattice, with comparability but not identity. Width, Height, and Diameter each specialize Length, and a kind may specialize several parents at once — a lattice, not a tree. Specialization is a preorder (reflexive and transitive) and induces a **one-way** coercion: a Quantity Width may be used where a Quantity Length is wanted, never the reverse, and that up-cast is a visible, deliberate loss of information. Width and Height remain **mutually comparable** — they share the super-kind Length — while staying distinct kinds. This requirement is now realized on the real standard: the ISO 80000-3 length family (width, height, distance, radius, ... of items 3-1.2 ... 3-1.12, all of dimension L) is specified as a specialization lattice over the general length kind, each species individuated **not by fiat but by an explicit measurement principle** (see the *ISO 80000-3* chapter).

R3 — General versus individual is type versus term. A kind is the general notion (a **type**); a particular measured value is an individual (a **term** of that type). Length is a type; the length of this pencil is a term of type Quantity Length. The two are never conflated — the punning OWL permits between a class and its instances is structurally impossible here.

1.2. Operation gating

R4 — Operations are gated by kind (the additive law). Arithmetic that keeps a quantity within its kind is well-typed; arithmetic across incompatible kinds is a **compile-time type error**, not a runtime check. $\text{Width} + \text{Width} = \text{Width}$ type-checks; $\text{Width} + \text{Height}$ does not — and, decisively, neither does $\text{Torque} + \text{Energy}$, even though torque and energy share a dimension. Matching dimension is not licence to add.

R5 — The interaction algebra is a partial, typed, ternary product. Some kinds combine **across** kinds to yield a third: torque times plane angle is energy, force times length is work. The combination is partial and curated — most pairs combine to **nothing** (fuel-consumption times rainfall is a category error, not a number) — and it carries a division dual, with multiplication and division proved inverse. This multiplicative combination is distinct from R4's additive, same-kind gate.

R6 — Operator availability is gated by the measurement scale, monotonically. Which operations even **exist** between two properties is fixed by their scale type, $\mathrm{nominal} \sqsubset \mathrm{ordinal} \sqsubset \mathrm{interval} \sqsubset \mathrm{ratio}$: a nominal property admits only $=$, a ratio property admits $\times$ and $\div$, and a richer scale licenses every operation a poorer one does. This gate is **orthogonal** to kind — kind says **which** properties may combine, scale says **which operators** are defined at all.

1.3. Soundness bridges

R7 — Dimension certifies coherence; it does not decide legality. Every kind has a dimension, but dimension is a **consistency check**, not the authorization: matching dimensions are necessary yet never sufficient — which is exactly what lets R4 reject $\text{Torque} + \text{Energy}$. Formally the dimension map $\dim : \mathrm{Kind} \rightarrow \mathrm{Dimension}$ is a **forgetful functor** and a **homomorphism**: $\dim(k_1 \cdot k_2) = \dim k_1 \cdot \dim k_2$.

In plain engineering terms: $\dim$ is a deliberately **lossy, one-way** translation from the rich kind layer down to the coarse SI-dimension layer. It keeps only the base-quantity exponents (length, mass, time, ...) and **forgets** everything else, so many distinct kinds collapse onto the same dimension and there is no way back — that is what "forgetful" names. Calling it a **homomorphism** says the translation **respects multiplication**: the dimension of a product equals the product of the dimensions (the exponents add), so the kind layer and the dimension layer can never disagree about what multiplying means. It is the discipline of ordinary dimensional analysis, stated and proved as a law rather than performed by hand.

R8 — Units are chosen values of a kind; conversion is a faithful round-trip. A unit is a distinguished value of a kind (a metre is a chosen length), and a measured quantity is a number times a unit. Converting a quantity from one unit to another of the **same** kind and back is the identity — conversion is multiplication by a ratio, defined only within a kind. There is no conversion between units of **different** kinds; that too is a type error, not a runtime check.

1.4. Aggregation

R9 — Extensive quantities aggregate additively over parts; intensive ones do not. For an **extensive** kind the value over a whole is the sum of the values over its disjoint parts — the mass of an assembly is the sum of its parts' masses. Where this fails it must **not** be assumed: volume on mixing is sub-additive (ethanol and water), and that negation is stated to keep the extensive predicate honest. Tracking which kinds are extensive is the precondition for soundly summing measurements.

1.5. Representation parametricity

R10 — A quantity value is parametric in its numeric representation type. The magnitude of a scalar quantity is carried at a **representation type** R , a type parameter bounded by whatever algebraic structure a given task demands, so the **same** kind-indexed value can be instantiated at whichever R the task needs — without changing the kind, scale, dimension, interaction, or extensivity layers above it. This is the axis dependent types make not just possible but **necessary**: the carrier the kind sits over is exactly where proof and execution diverge, and a single quantity must serve both. Three representations matter, and they are not interchangeable:

- **Lean's $\mathbb{R}$ — for proofs.** A complete ordered field: the continuous specification against which laws (monotonicity, conversion round-trips, extensivity) are stated and proved.
- **A finite floating-point rounding model — for numerical-accuracy reasoning.** TorchLean's FP32 (NeuralFloat at binary32 precision: round-to-nearest, ties-to-even over $\mathbb{R}$, finite-only, with `toReal : FP32 → ℝ`). It is **non-computable** — it exists to bound rounding error against the $\mathbb{R}$ spec, not to run.
- **An executable IEEE-754 type — for exact-execution reasoning and code generation.** TorchLean's IEEE32Exec (a bit-level UInt32 binary32 kernel) that models NaN, infinities, signed zeros, and subnormals, with `toFloat/ofFloat` and `isNaN`. This is the representation that actually runs and the one in which exceptional values (NaN) can be reasoned about — what one might call the "exec" float, distinct from the FP32 rounding spec.

Soundness **across** representations is itself a theorem, not an assumption: an exec/spec **refinement** (the executable float result is the rounding of the real result) lets a law proved over $\mathbb{R}$ transfer to the executable run. In plain engineering terms: choose the number type to fit the job — reals to prove, the rounding model to bound error, the IEEE kernel to run and to catch NaN — while the kind, dimension, and unit machinery above is written once and is identical for all three. This refines R1/R3/R4's kind-indexed value into a doubly-indexed `Quantity (k : KindOfProperty) (R : Type)` whose arithmetic is bounded by a `Carrier` typeclass on R . The value layer and its additivity laws are **realized** (sorry-free): `Quantity k R` with same-kind addition, the additivity laws proved once over any lawful carrier, and three carriers — `Int` and $\mathbb{R}$ (lawful, for proof) and `Float` (executable, deliberately not lawful). The exec/spec **refinement** across representations is now **also realized** (sorry-free): the abstract bridge `CarrierRefinement` and its kind-indexed capstone `Quantity.add_refines` — a law over the lawful spec

carrier descends to the exec carrier as one rounding step — instantiated in the separately-built Torch library at TorchLean's binary32: FP32 (the rounding **spec**, an **unconditional** refinement of $\mathbb{R}$) and IEEE32Exec (the **executable** kernel, refining $\mathbb{R}$ on the finite/no-overflow path, with overflow surfaced as an explicit side condition rather than silently dropped).

1.6. Value representation: vectors and scalar units

R11 — Units are scalar; a vector quantity is a numerical array times one scalar unit. Following ISO 80000-2 §18 (scalars, vectors and tensors), a vector (or tensor) quantity is written as a **numerical** vector multiplied by a single unit, and "all units are scalars" — **not** as a collection of per-coordinate quantity values each carrying its own number $\times$ unit; the quantity itself is independent of the coordinate system while its numerical components are not. This is exactly the R10 carrier taken at a numerical-array type: a vector quantity is $\text{Quantity } (k : \text{KindOfProperty}) \text{ } (\text{Fin } n \rightarrow \mathbb{R})$ — one kind k , the numbers an n -vector in the carrier $\mathbb{R}$, and one **scalar** *MetrologicalUnit* of the kind for the whole vector — and the additivity laws transfer to the vector carrier by the **same** parametric proof used for scalars (the pointwise $\text{Carrier } (\text{Fin } n \rightarrow \mathbb{R})$). It is the ISO 80000-2 §18 **numerical-array** reading of value representation, and it aligns with the VIM's "a quantity is scalar; a vector/tensor is a composite of scalar quantities" — the priority SysML v2 inverts (see **Why a calculus, not a taxonomy**). The fuller **structural** apparatus is a separate axis, still owed below.

1.7. Verified classification: kind-laws that instantiate at the quantity level

R12 — A classification is a certificate, not an assertion; kind-laws instantiate at the quantity level. By default a kind is a **tag**: $\text{Quantity } k \ \mathbb{R}$ records the kind but the magnitude is arbitrary, so a value can be labelled with a kind it has not earned. R12 upgrades a classification to a **certificate** — a proof that a quantity satisfies its kind's **defining relation** (the formalized ISO 80000 **Remark**) to quantities of the kinds it is built from. A length built as speed $\times$ time is certified by construction; a bare (999) cannot be certified against those factors. The kind becomes a **refinement** (a tag plus its earned proof).

The reason this strengthens the calculus is **instantiation**. Every kind-law is stated as a carrier-parametric universal over quantities with the kind-relation as a premise, so it instantiates to a quantity-level fact by ordinary application — the inter-kind generalization of R10's carrier-parametric laws — and any property of a kind's defining relation transports to **every** quantity certified under it (every certified surface area is ≥ 0 , because the area element is). The pattern comes in three algebraic shapes — **product** (area = length $\cdot$ length, energy = force $\cdot$ length), **quotient** (speed = length / duration, plane angle = arc / radius), and **reciprocal** (frequency = 1 / period, curvature = 1 / radius) — each a kind-law family in the core; analysis-shaped relations such as area as a surface integral $\iint \sqrt{g} \, du \, dv$ follow the same discipline in the layer where their mathematics lives. These are exercised directly on ISO 80000-3's own **Remarks** (see the *ISO 80000-3* chapter), where, for instance, the plane angle is **computed** to be dimension one because it is a ratio of two lengths. R12 unifies the two halves of the

standards work: the formalized remark **is** the defining relation, and classification is the witnessed instantiation of it.

1.8. Out of scope (for now)

Value representation in the structural sense — coordinate frames, tensor variance (the covariant/contravariant split), bound-versus-free vectors, and frame transformations — is **not yet specified** here, beyond the ISO 80000-2 §18 numerical-array reading now specified as R11. This **structural** axis is distinct from R10's **numeric carrier** R and R11's **numerical array**. It is a genuine systems-engineering requirement, but a **separate, orthogonal** axis: its principled home is an index **over** the kind, never a layer the kind hangs beneath (the inversion the **Why a calculus, not a taxonomy** section charges against SysML v2). Stating it as owed keeps the boundary honest.

1.9. The requirements at a glance

Requirement	Concrete test	Specified as	Status
R1 — kind discrimination within a dimension	$vwc \neq gwc$, both dimension one	kind-indexed Quantity $k R$; dimension-1 disambiguation	proved
R2 — specialization lattice + comparability	Width, Height, Diameter specialize Length (ISO 80000-3); weight, static vs kinetic friction force specialize Force (ISO 80000-4) — comparable yet distinct, by examination principle	Specializes preorder; MutuallyComparable; examination defining-aspect (Refines, distinct_of_examPrinciple)	proved
R3 — general vs individual (type vs term)	Length a type; this pencil's length a term	KindOfProperty vs Quantity $k R$	proved
R4 — kind-gated addition	Width+Width ok; Width+Height, Torque+Energy rejected	Quantity.add over Quantity $k R$ (same-kind gate)	proved
R5 — interaction algebra (partial, typed)	torque $\times$ angle = energy; fuel $\times$ rainfall an error	KMul / KDiv; multiplication–division inverse	proved
R6 — scale-type operator availability (monotone)	nominal: only =; ratio: $\times$, $\div$	ScaleType order; operator monotonicity	proved
R7 — dimension certifies, not decides	dim many-to-one; necessary $\neq$ sufficient	dim homomorphism	proved
R8 — units and conversion round-trip	round-trip metre/foot conversion is the identity; cross-kind is a type error	Unit k ; conversion round-trip	planned
R9 — extensive aggregation	mass sums over parts; volume-on-mixing does not	Extensive; additive law + counterexample	proved
R10 — numeric representation parametricity	same value at $\mathbb{R}/\text{Int}$ (proof), Float	Carrier-bounded Quantity $k R$; parametric additivity laws;	proved (layer + laws

Requirement	Concrete test	Specified as	Status
	(executable); FP32/ IEEE32Exec refine $\mathbb{R}$	CarrierRefinement + Quantity.add_refines	+ exec/spec refinement)
R11 — scalar units; vector = numerical array $\times$ one scalar unit (ISO 80000-2 §18)	a displacement as Quantity k ($\text{Fin } 3 \rightarrow \mathbb{R}$) with one metre; laws transfer	pointwise Carrier ($\text{Fin } n \rightarrow \mathbb{R}$); scalar MetrologicalUnit	proved
R12 — verified classification; kind- laws instantiate at the quantity level	a length built as speed $\times$ time is certified by construction; (999) cannot be certified; every certified surface area is $\geq$ 0	ProductKind kind-laws; Quantity.IsProduct certificate + smart constructor; defining relations from the ISO 80000 remarks (area = $\iint \sqrt{g}$)	proved (product family + area instance)
(out of scope) structural value representation	coordinate frames, tensor variance, transforms	an orthogonal index over the kind	not specified

2. Why a calculus, not a taxonomy

PropertyKindCalculus continues a line of machine-checkable metrology modeling, and inherits its problem statement from exactly where that line stops.

2.1. From QUDV to an OML metrology vocabulary

OMG SysML 1.2 introduced [QUDV](https://www.omgwiki.org/OMGSysML/doku.php?id=sysml-qudv:quantities_units_dimensions_values_qudv) (https://www.omgwiki.org/OMGSysML/doku.php?id=sysml-qudv:quantities_units_dimensions_values_qudv) (Quantities, Units, Dimensions, Values) as a library for declaring quantity kinds, units, and their dimensional factoring; QUDV's foundations were then revised substantially across SysML 1.3 and 1.4.

In parallel, a formalization in OML — leveraging the semantics of OWL2-DL for classification-based reasoning — reworked those foundations, the quantity-kind and unit *stereotypes* carried by SysML, to align them with Dybkær's *An Ontology on Property*. That effort became the [opencaesar metrology vocabulary](https://github.com/opencaesar/metrology-vocabularies) (<https://github.com/opencaesar/metrology-vocabularies>), whose current version combines the updates and motivations of the VIM4 2nd Committee Draft (2CD, 2023-07-31) with Dybkær's ontological grounding. OWL2 (SROIQ) carries the metrology *taxonomy* well: specialization, the general-vs-individual quantity distinction (encoded as concept vs. instance), dimensional factoring, and OWL2-DL/SPARQL consistency checks over units.

Where a description logic stops is visible in that vocabulary's own *dimensional analysis*: computing a derived quantity's dimension from its factors is an arithmetic induction that has to be pushed out of OWL2 into SPARQL 1.1 Update, and even there it is brittle — updates report success while inserting no triples, because "update succeeded" is not a statement about *what was established*. Dimension arithmetic sits right at that boundary; the kind interaction algebra and the algebraic *laws* are past it.

2.2. What a description logic cannot express

A description logic captures the *taxonomy* but is structurally unable to capture the *calculus*. The recurring gap is arithmetic, operations, definedness-conditions, laws, and proofs:

- **Dimension arithmetic** ($\dim(a \cdot b) = \dim a + \dim b$): OWL2 has no datatype arithmetic; in Lean it is an ordinary function with a homomorphism theorem.
- **The interaction algebra** (torque $\times$ angle = energy): role chains are binary, regular, and carry no arithmetic side-conditions; in Lean it is a ternary relation with a dimensional-coherence proof obligation.
- **Scale-type operator gating** (which of $=, <, +, \times$ are **defined**): DL has no operations; in Lean the scale indexes which typeclasses apply, and monotonicity of availability is a proved meta-theorem.
- **Extensivity** ($\mathrm{value}(\mathrm{whole}) = \sum \mathrm{value}(\mathrm{parts})$): no quantified arithmetic in DL; in Lean it is a $\forall$ -theorem.
- **Units** (a unit is a chosen value of a kind; conversion is a ratio): no arithmetic in DL; in Lean it is a definition with a round-trip theorem.
- **Meta-theorems** (specialization is a preorder; $\dim$ is a homomorphism): a DL reasoner checks **consistency**, not **laws** — proving the laws is the point.

2.3. SysML v2: a quantity model organized by value representation

OMG's current generation ships a standard-library treatment of quantities and units, examined here at tag [SysML v2, 2026-04](https://github.com/Systems-Modeling/SysML-v2-Release/tree/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units) (<https://github.com/Systems-Modeling/SysML-v2-Release/tree/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units>) (its Quantities, MeasurementReferences, and ISQ packages). Read against Dybkær, it is built around a different organizing axis than the one taken here as primary, and the contrast is the cleanest way to say what a **kind-primary** model buys.

2.3.1. The evidence: representation is the root, kind is a leaf

The quantity-value hierarchy is rooted in **value representation**, not in quantity kind. The most general value is `TensorQuantityValue`, a tensor of any order, and the specialization chain runs **toward** the

scalar, each more specific type being a lower-order tensor — `VectorQuantityValue` $\Rightarrow$ `TensorQuantityValue`, then `ScalarQuantityValue` $\Rightarrow$ `VectorQuantityValue` ([SysML v2, 2026-04 — Quantities.sysml, lines 17–46](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/Quantities.sysml#L17-L46) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/Quantities.sysml#L17-L46>)): `\text{Scalar}` $\sqsubseteq$ `\text{Vector}` $\sqsubseteq$ `\text{Tensor}`. So a scalar **is a** tensor of order 0 and a vector **is a** tensor of order 1. What a value's type primarily records is the apparatus of that representation:

- its tensor order,
- the split of that order into **contravariant** and **covariant** indices — the attributes `contravariantOrder` and `covariantOrder` on `TensorQuantityValue`, constrained to sum to the order ([SysML v2, 2026-04 — Quantities.sysml, lines 33–36](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/Quantities.sysml#L33-L36) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/Quantities.sysml#L33-L36>))
- whether a vector is bound or free, and
- a matching measurement reference (tensor / vector / scalar) carrying the coordinate frame and its transformations.

Each of these is a statement about **how a quantity is measured, coordinatized, and valued**, not about **what kind of property** it is. The contravariant/covariant split is the sharpest case of this representation-centric view — the one record-field that is **unambiguously** representational, on three escalating counts.

1. **Pure coordinate bookkeeping.** It records only how a value's components transform under a change of frame — nothing about what the quantity **is**. The other three fields each record a facet of the quantity itself:
 - tensor order tracks its mathematical character,
 - bound-versus-free is a genuine physical distinction (and one the library actually uses), and
 - the measurement reference names a dimension and a frame.

The variance split alone carries no information about the quantity kind.

2. **Not quantity-invariant.** Using the metric to raise a covariant (lower) index or lower a contravariant (upper) one moves a unit between the two counts — say from `contravariantOrder` 2, `covariantOrder` 0 to a mixed 1 and 1 — while the order, and the physical quantity, stay fixed. That the quantity is unchanged is a fact of **tensor algebra** — the metric's raising/lowering isomorphism — not a SysML v2 statement: the library defines no raise- or lower-index operation, so it would record the two splits as distinct `TensorQuantityValues` with nothing to mark them as one quantity. A feature that a pure change of representation can move while the quantity is held fixed is thus a property of the representation, not of the quantity's kind. Hold the quantity fixed and the other three fields are pinned with it — order, bound-status, and reference do not change — so the variance split is the one field free to vary while the modeled quantity stays put (worked through just below).

3. **Empirically inert.** The split is declared on the abstract root and named by its constraint, yet assigned by **no** quantity anywhere in the library: even the genuine order-2 kinds — Cartesian3dStressTensor, with strain and moment of inertia alongside it — inherit the two slots and leave them unset, fixing only `isBound`, `num`, and `mRef` ([SysML v2, 2026-04 — ISQMechanics.sysml, lines 730–746](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/ISQMechanics.sysml#L730-L746) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/ISQMechanics.sysml#L730-L746>)). The apparatus is carried at the root whether or not any kind exercises it. By contrast `isBound` **is** used (88 free against 7 bound), so the point is precise rather than blanket: bound-versus-free is live machinery, while the variance split is declared-but-unexercised overhead inherited from the tensor root.

To see why the second count bites — and what the model **loses** by it — follow the library's own machinery. Cartesian3dStressTensor stores its value as `num: Real[9]`, nine bare reals, with the variance split left blank (same listing). In an orthonormal Cartesian frame that omission is harmless: under the Euclidean metric the contravariant components σ^{ij} , the mixed $\sigma^i{}_j$, and the covariant σ_{ij} are numerically equal, so it does not matter which the nine numbers are. But `TensorCalculations::transform` is defined to carry a tensor value from a source frame to a target frame through a `CoordinateTransformation` ([SysML v2, 2026-04 — TensorCalculations.sysml, lines 45–49](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/TensorCalculations.sysml#L45-L49) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/TensorCalculations.sysml#L45-L49>), [MeasurementReferences.sysml, lines 126–135](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/MeasurementReferences.sysml#L126-L135) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/MeasurementReferences.sysml#L126-L135>)), and the transformation rule is keyed on the split: a contravariant index transforms by the Jacobian of the coordinate change, a covariant index by its inverse — mutually inverse rules, so the two generally yield different numbers. Move to a frame where they differ — any non-orthonormal frame, which is to say the everyday cylindrical, spherical, or material coordinates of continuum mechanics — and the nine numbers transform by **different** rules depending on a datum the model declared but never filled in.

What is missing, then, is the ability to transform an order-2 (or higher-order) quantity between frames **at all**: the `transform` contract cannot be met without the split, so for exactly the quantities where it would matter the operation is underdetermined. And the gap is **silent**. The concrete types are pinned to a Cartesian frame, where the omission never shows; the model type-checks and looks complete, and fails only where one would actually rely on it — in the curvilinear frames it was never exercised in. That is precisely a representational hole that validation cannot see: correct on every case anyone tested, wrong on the first case anyone needed.

The lone constraint the library attaches to the split sharpens the reading. `orderSum` requires `contravariantOrder + covariantOrder == order` ([SysML v2, 2026-04 — Quantities.sysml, line 36](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/Quantities.sysml#L36) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/Quantities.sysml#L36>)). Metrologically it is a well-formedness invariant, not a statement about the quantity: it says only that the chosen split must be a valid partition of the quantity's actual order. Its content is that **order is the frame-invariant datum and the split is a convention that must respect it** — raising or lowering an index slides one unit between `covariantOrder` and `contravariantOrder` but holds their sum, the order, fixed. So `orderSum` is exactly what makes **the same quantity, a different split** formally coherent: the order — which carries the quantity's mathematical character — is conserved while the

representation changes underneath it. It constrains nothing metrological: not dimension, unit, scale, or kind, only that the coordinate bookkeeping adds up. The one rule SysML v2 writes about variance is thus itself a pure-representation consistency check — the section's thesis in miniature.

The kind-of-quantity sits one level **down**, as a leaf, and the source is decisive on this point: there is no first-class "kind" entity. Length, mass, temperature, and the other base quantities are each declared as a **subtype of the scalar value type** — attribute `def LengthValue :> ScalarQuantityValue` — with the dimension carried only by the corresponding unit's `quantityDimension`, e.g. `LengthUnit` ([SysML v2, 2026-04 — ISQBase.sysml, lines 16–38](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/ISQBase.sysml#L16-L38) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/ISQBase.sysml#L16-L38>)), and that `quantityDimension` attribute itself lives on `ScalarMeasurementReference` ([SysML v2, 2026-04 — MeasurementReferences.sysml, lines 82–99](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/MeasurementReferences.sysml#L82-L99) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/MeasurementReferences.sysml#L82-L99>)). In other words, "length-the-kind" exists only as (i) a nominal subtype hanging beneath `ScalarQuantityValue` and (ii) a power-product of base quantities on its unit — never as a property whose nature is named in its own right. To reach a kind one must already have committed to a representation branch: the kind is **nested under** the representation. The proportions confirm the weight — across the ISQ library 319 kinds are declared on the scalar branch, against 2 vector, 4 tensor, and 51 fixed-three-vector kinds.

That places the concern precisely. A valued quantity has two independent classification axes:

- **(A) kind-of-property** — its nature, fixed by Dybkær's examination principle, common to all mutually comparable quantities;
- **(B) value representation** — scalar / vector / tensor order, coordinate frame, bound-versus-free — what SysML v2 packages as the measurement reference.

Dybkær's ontology lives entirely on axis (A) and is deliberately **agnostic** to (B): whether a position is recorded as one number, three Cartesian components, or a tensor in a chosen frame does not change **what kind of property** it is. SysML v2 makes axis (B) the root and nests (A) beneath it. The model even flags the departure from its own source: it quotes the VIM3 "quantity" Note 5 — "A quantity as defined here is a scalar. However, a vector or a tensor, the components of which are quantities, is also considered to be a quantity" — then notes that "the rest of [VIM] does not explicitly define how tensor and vector quantities can be or should be supported" ([SysML v2, 2026-04 — MeasurementReferences.sysml, lines 29–31](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/MeasurementReferences.sysml#L29-L31) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/MeasurementReferences.sysml#L29-L31>)). The tensor-rooted hierarchy is thus SysML's own interpretation, and it **inverts** the VIM's priority — both the quoted VIM3 and the VIM4 2CD (quantity, 1.1 Note 4: "As defined here, quantities are scalar ... vectors, and tensors are considered quantities in a broader sense if their components are (scalar) quantities") make the quantity fundamentally scalar, with vectors and tensors as composites **of** scalar quantities; SysML v2 makes the tensor primary and the scalar the degenerate order-0 case.

2.3.2. How PropertyKindCalculus takes the kind as root instead

PropertyKindCalculus inverts the nesting: axis (A) is the root. A kind-of-property is a first-class value (`KindOfProperty`), and a quantity is indexed **by its kind** — `Quantity (k : KindOfProperty)` — so the kind is fixed before, and independently of, any choice of how the value is recorded. Dimension is not the organizing principle but a forgetful functor $\dim : \mathrm{Kind} \rightarrow \mathrm{Dimension}$ that **certifies** coherence without **deciding** what is allowed (developed in the next section). Two consequences fall out directly: two kinds that share a dimension remain distinct types — so the dimension-1 disambiguation is expressible — and an operation across kinds is a type error rather than a silent success. Value representation — the axis SysML v2 puts at the root — is, candidly, **not yet specified** here; its principled home is an **orthogonal** index over the kind, never a layer the kind hangs beneath.

2.3.3. The two designs side by side

Aspect	SysML v2 (2026-04)	PropertyKindCalculus
Root organizing axis	value representation (tensor order, frame)	kind-of-property (Dybækær)
A kind of quantity is...	a leaf subtype of <code>ScalarQuantityValue</code>	a first-class type index, <code>Quantity (k : KindOfProperty)</code>
Role of dimension	the discriminator for commensurability, on the reference	a forgetful functor <code>dim</code> ; certifies, never decides
Two kinds, one dimension (vwc vs gwc)	indistinguishable	distinct by construction
Adding a length to a mass	well-typed (<code>Scalar × Scalar → Scalar</code>)	a type error
Scalar / vector / tensor, frames, transforms	first-class and rich	not yet specified (owed — an orthogonal index)
Algebraic laws	a modelling library; none machine-checked	proved theorems (preorder, homomorphism, monotonicity)

The table is honest in both directions: SysML v2's representation-first design buys real, first-class machinery for coordinate frames, transformations, and stress and strain tensors that systems engineering genuinely needs — machinery PropertyKindCalculus does not yet have. The claim is not that one model is right; it is that the axes are **orthogonal and should not be nested**.

2.3.4. Limitations of the SysML v2 approach

Rooting the hierarchy in representation has three consequences that bear directly on PropertyKindCalculus's problem statement:

1. **A kind cannot be named without first choosing a representation, so one kind fragments.** Position is a single kind-of-property, yet appears as CartesianPosition3dVector, CylindricalPosition3dVector, SphericalPosition3dVector, and PlanetaryPosition3dVector — individuated by **coordinate frame**, not by the nature of the property. The cylindrical and spherical forms even carry mixed-dimension components (a length and two angles), so the single documented "quantity dimension L^1 " does not survive its own representations ([SysML v2, 2026-04 — ISQSpaceTime.sysml, lines 304–353](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/ISQSpaceTime.sysml#L304-L353) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/ISQSpaceTime.sysml#L304-L353>)).
2. **The within-dimension conflation is left untouched.** On the scalar branch the dimension is the only discriminator, so volumetric and gravimetric water content, relative permittivity, reflectivity, and emissivity remain indistinguishable — exactly the confusion that motivates PropertyKindCalculus. Putting representation at the root does nothing about it; the two are orthogonal.
3. **The calculus is kind- and dimension-blind.** Scalar addition is declared $\text{ScalarQuantityValue} \times \text{ScalarQuantityValue} \rightarrow \text{ScalarQuantityValue}$, so adding a length to a mass is well-typed; nothing gates an operation on agreeing kind or dimension ([SysML v2, 2026-04 — QuantityCalculations.sysml, line 29](https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/QuantityCalculations.sysml#L29) (<https://github.com/Systems-Modeling/SysML-v2-Release/blob/2026-04/sysml.library/Domain%20Libraries/Quantities%20and%20Units/QuantityCalculations.sysml#L29>)).

2.4. How the calculus maps into Lean

Two things in particular resist a DL encoding: the *interaction algebra* (which kind $\times$ which kind = which kind, a *partial* relation with arithmetic side-conditions, not a role chain), and the difference between *instantiation* and *subtyping* (that a radius *is* a length, versus that this pencil's length *is an instance of* the length kind — conflated by punning in OWL). In Lean both map cleanly, and the mapping is the design:

1. **Kinds and units are type-level parameters.** `Quantity (k : QuantityKind) (u : Unit) : Type` — a kind and a unit index a *type*, not a value of one universal Quantity class. This is the move OWL cannot make.
2. **Specialization ($\sqsubseteq$) induces coercions between types.** It lives on those parameters: `Quantity Radius metre → Quantity Length metre` is sound — a radius *is* a length — and the reverse is not. Up-casting to a parent kind is an explicit, visible loss of information.
3. **Instantiation is term-of-type, not subtyping.** The length of a pencil is a *term* `pencil : Quantity Length metre := ⟨5.2⟩` — a value of that type, related to its kind by typing/instantiation, never by subtyping. The general/individual distinction *is* the type/term distinction.

4. **The kind algebra is a partial typed algebra, not a group.** PhysLib's `Dimension` is a *total* `CommGroup` — multiply, divide, invert anything. Kinds are not: `Torque × Angle = Work` is meaningful; `Torque + Energy` must be *blocked* even though the dimensions match; `FuelConsumption × Rainfall` is an error (Flater's own example). The meaningful products are encoded as a relation surfaced through typeclass resolution — Flater's "map kinds onto classes + operator overloading," lifted into Lean's elaborator.
5. **Dimension is a soundness functor, not the primary type.** In QUDV and most quantity libraries dimension is the *primary* classifier: two quantities interoperate exactly when their dimensions agree — which is precisely the move that collapses every dimension-one quantity into one indistinguishable type. `PropertyKindCalculus` inverts the dependency: the *kind* is the primary type-level index, and dimension is recovered as a forgetful functor $\backslash \mathrm{dim} : \backslash \mathrm{Kind} \rightarrow \backslash \mathrm{Dimension}$. Its job is *certification*, not decision — the coherence theorem $\mathrm{KMul} \ k_1 \ k_2 \ k_3 \rightarrow \mathrm{dim} \ k_3 = \mathrm{dim} \ k_1 \cdot \mathrm{dim} \ k_2$ proves every product the kind algebra admits is dimensionally consistent — but `dim` never decides whether an operation is *allowed*; the kind relation does. Agreeing dimensions are necessary, not sufficient.
6. **Specialization ($\sqsubseteq$) is a preorder — a lattice, not a tree.** Flater stresses this (citing Formal Concept Analysis): a kind can specialize several parents. Unit-1 subtyping (Flater §6) is then just the *dimension-1 slice* of that kind lattice — and nothing forces it to be uniform: units of plane angle, for one, could carry their own subtyping reflecting the different measurement principles by which they are realized.
7. **The numeric carrier is a type parameter, bounded by a typeclass (R10).** A scalar quantity is `Quantity (k : KindOfProperty) (R : Type)` with the arithmetic on `R` supplied by a `Carrier` typeclass — the kind fixes **what is measured**, `R` fixes **in what numbers**. Everything above `R` (kind discrimination, scale gating, the dimension functor, the interaction algebra, extensivity) is written once, against the typeclass, and is reused verbatim at every `R` (the additivity laws, for one, are proved a single time over any lawful carrier and hold at $\mathbb{R}$ and `Int` with no per-carrier proof). This is the architecture TorchLean already runs: models are written against a `Context α` typeclass (arithmetic + transcendentals
 - comparison) and instantiated at $\alpha := \mathbb{R}$ for the **spec**, $\alpha := \text{FP32}$ for the finite **rounding** model, and $\alpha := \text{IEEE32Exec}$ for the **executable** IEEE-754 kernel — the **same** source elaborated at three carriers. The kind layer here is the missing index **above** that carrier: TorchLean makes a tensor polymorphic in its scalar type; `PropertyKindCalculus` makes the scalar polymorphic in its **kind**, and the two indices compose. Crucially the carriers are bridged by **refinement theorems** (`toReal (op_exec x) = round (op_real (toReal x))`), so a law proved at $\mathbb{R}$ is not stranded in the proof world — it descends to the float that runs. A description logic has no type parameters and no typeclasses, so this whole axis is unavailable to it; here it is the ordinary discipline of parametric polymorphism.

The trigger was concrete. PhysLib's `Dimension` makes all dimension-one quantities equal, so it cannot distinguish volumetric ($\mathrm{L}^3/\mathrm{L}^3$) from gravimetric (M/M) soil moisture, nor either from

permittivity, reflectivity, or emissivity. Every dangerous soil-moisture confusion is dimension one. Indexing quantities and units by **kind** is what restores the distinction — this is the dimension-1 disambiguation capstone in the Units chapter.

3. How to read the status

Nodes that link a real declaration with (`lean := "PropertyKindCalculus...."`) report their **proved** status straight from the checked, sorry-free core. Nodes tagged `planned` carry an informal statement and a proof sketch only; they show as in-progress goals until formalized. The headline deliverables are tagged `capstone`.

4. The Proved Spine

This chapter records what `PropertyKindCalculus` already establishes, sorry-free, in its Mathlib-free core. Every node here links to a real declaration with (`lean := "PropertyKindCalculus...."`), so its *proved* status in the dependency graph is read directly from the checked library rather than asserted in prose.

The point to keep in view: each of these is a law or an operation, not a class membership. A description-logic reasoner (OWL2/SROIQ) can record that `width` specializes `length`; it cannot state, let alone prove, that specialization is transitive or that operator availability is monotone in scale. Those are the entries below.

4.1. Foundations — system and object (Dybæk Ch. 3)

Definition

4.1.1.

uses 0

statement">used

A *system* (Dybæk §3.3) is a demarcated arrangement of elements and their relationships; *object* (§3.3 Note 6) is given as a synonym. Specified abstractly by identity here; the mereological structure that extensivity needs is added in a later module.

Lean code for Definition

4.1.1.

1 definition

structure (1 field) · defined in PropertyKindCalculus/ Foundations.lean

complete

```
structure PropertyKindCalculus.System : Type
```

Proof for Definition 4.1.1

uses 0

A one-field structure with `DecidableEq`; nothing to prove. The synonym `Object := System` keeps the instance layer reading as "characterizes an object".

4.2. Scale — the operator-based division of ⟨property⟩ (Dybækær Ch. 12)

Definition 4.2.1.

group

uses 0

used by 3

✓ L3

The four *scale types* of Dybækær Fig. 12.21, ordered by operator richness: $\mathrm{nominal}$ $\sqsubseteq$ $\mathrm{ordinal}$ $\sqsubseteq$ $\mathrm{interval}$ $\sqsubseteq$ ratio , admitting respectively $\{=, \neq\}$, then $\{<, >\}$, then $\{+, -\}$, then $\{\times, \div\}$.

Lean code for Definition 4.2.1.

1 definition

complete

```
inductive (4 constructors) · defined in PropertyKindCalculus/Scale.lean
```

```
inductive PropertyKindCalculus.ScaleType : Type
```

Proof for Definition 4.2.1

uses 0

A four-constructor inductive with a rank : ScaleType → Nat and the induced $\leq$. The lemmas `le_def`, `le_refl`, `le_trans`, and `le_antisymm` establish that $\leq$ is the expected linear order.

Theorem 4.2.2.

group

statement" uses 10

✓ L3

Owner

Tags

Nicolas Rouquette

proved

The scale order is a linear order: reflexive (`le_refl`), transitive (`le_trans`), and antisymmetric (`le_antisymm`) on [the scale types](#).

Lean code for Theorem 4.2.2.

1 theorem

complete

```
theorem · defined in PropertyKindCalculus/Scale.lean
```

```
theorem PropertyKindCalculus.ScaleType.le_antisymm  
  {a b : PropertyKindCalculus.ScaleType} (h1 : a ≤ b) (h2 : b ≤ a) :  
  a = b
```

Proof for Theorem 4.2.2

uses 0

Reflexivity and transitivity are inherited from `Nat.le` on `rank`. Antisymmetry reduces to `Nat.le_antisymm` on the ranks followed by a four-by-four case split (`cases a <=> cases b <=> simp_all`).

Theorem

4.2.3.

group

statement" uses 11

Owner

Nicolas Rouquette

Priority

high

Tags

capstone

proved

Operator-availability monotonicity. A richer scale licenses every operation a poorer one does: if $a \leq b$ then *AllowsOrder*, *AllowsDifference*, and *AllowsRatio* each transport from a to b . This is a meta-theorem OWL2's reasoner cannot even phrase, since it quantifies over which operations are defined for a kind. Builds on [the scale types](#).

Lean code for Theorem 4.2.3.

1 theorem

theorem · defined in PropertyKindCalculus/
Scale.lean

complete

```
theorem PropertyKindCalculus.ScaleType.allows_mono
  {a b : PropertyKindCalculus.ScaleType} (h : a ≤ b) :
  (a.AllowsOrder → b.AllowsOrder) ∧
  (a.AllowsDifference → b.AllowsDifference) ∧
  (a.AllowsRatio → b.AllowsRatio)
```

Proof for Theorem 4.2.3

uses 0

Decide all sixteen ordered pairs: cases a <;> cases b <;> simp_all [AllowsOrder, AllowsDifference, AllowsRatio, le_def, rank].

4.3. Kind-of-property and kind-of-quantity (Dybækær Ch. 6, 13)

Definition

4.3.1.

group

statement" uses 11

✓ L

A *kind-of-property* (§6.19) bundles a terminological id, a scale (from [the scale types](#)), and an optional examination principle. A kind is a *kind-of-quantity* (§13.3.1) iff its scale has magnitude (is at least ordinal).

Lean code for Definition 4.3.1.

1 definition

structure (3 fields) · defined in PropertyKindCalculus/
Kind.lean

complete

```
structure PropertyKindCalculus.KindOfProperty : Type
```

Proof for Definition 4.3.1

uses 0

A structure with DecidableEq, IsQuantity, and the scale divisions IsNominal/IsOrdinal/IsDifferential/IsRational (§13.2), are definitional predicates over the scale field.

Theorem 4.3.2.

group

statement" >us

Owner

Tags

Nicolas Rouquette

proved

A nominal kind is, by construction, not a kind-of-quantity: it has no magnitude. Uses [kind-of-property](#).

Lean code for Theorem 4.3.2.

1 theorem

theorem · defined in PropertyKindCalculus/
Kind.lean

complete

```
theorem PropertyKindCalculus.KindOfProperty.nominal_not_quantity  
  {k : PropertyKindCalculus.KindOfProperty} (h : k.IsNominal) :  
  ¬k.IsQuantity
```

Proof for Theorem 4.3.2

uses 0

Unfold IsQuantity, IsNominal, and HasMagnitude; the scale is nominal, so scale ≠ nominal is false.

Theorem 4.3.3.

group

statement" >us

Owner

Tags

Nicolas Rouquette

proved

A rational kind is a kind-of-quantity. Uses [kind-of-property](#).

Lean code for Theorem 4.3.3.

1 theorem

theorem · defined in PropertyKindCalculus/
Kind.lean

complete

```
theorem PropertyKindCalculus.KindOfProperty.rational_isQuantity  
  {k : PropertyKindCalculus.KindOfProperty} (h : k.IsRational) :  
  k.IsQuantity
```

Proof for Theorem 4.3.3

uses 0

The scale is ratio, so scale ≠ nominal holds; unfold and simplify.

4.4. Specialization and mutual comparability (OML / Dybkær §6)

Definition4.4.1.

groupstatement"uses"6

Specialization Specializes E is the reflexive–transitive closure of a direct-parent edge relation E over [kind-of-property](#). Different E give different per-application hierarchies — the extensibility a fixed inductive would deny.

Lean code for Definition4.4.1.

1 definition

```
inductive (2 constructors, Prop, 3 parameters) · defined in PropertyKindCalculus/ Specialization.lean

inductive PropertyKindCalculus.Specializes
  (E :
    PropertyKindCalculus.KindOfProperty →
    PropertyKindCalculus.KindOfProperty → Prop) :
  PropertyKindCalculus.KindOfProperty →
  PropertyKindCalculus.KindOfProperty → Prop
```

Proof for Definition 4.4.1

uses 0

An inductive with `refl` and `step` constructors; `of_edge` lifts a single edge.

Theorem4.4.2.

groupstatement"uses"0

Owner	Priority	Tags
Nicolas Rouquette	high	capstone proved

Specialization is a preorder. It is reflexive (the `refl` constructor) and transitive (`Specializes.trans`). Stating — let alone proving — an algebraic law of the taxonomy is exactly what an OWL2 reasoner cannot do; it checks consistency, not laws. Uses [specialization](#).

Lean code for Theorem4.4.2.

1 theorem

```
theorem · defined in PropertyKindCalculus/ Specialization.lean

theorem PropertyKindCalculus.Specializes.trans
  {E :
    PropertyKindCalculus.KindOfProperty →
    PropertyKindCalculus.KindOfProperty → Prop}
```

```

{a b c : PropertyKindCalculus.KindOfProperty}
(h1 : PropertyKindCalculus.Specializes E a b)
(h2 : PropertyKindCalculus.Specializes E b c) :
PropertyKindCalculus.Specializes E a c

```

Proof for Theorem 4.4.2

uses 0

Transitivity is by induction on the first derivation: `refl` returns the second derivation, `step` re-applies the constructor under the inductive hypothesis.

Definition 4.4.3.

group

statement"uses 2

✓ L3

Two kinds are *mutually comparable* iff they share a common super-kind ($\exists p, a \sqsubseteq p, b \sqsubseteq p$). This is how width and height stay comparable as lengths while remaining distinct sub-kinds. Uses [specialization](#).

Lean code for Definition 4.4.3.

1 definition

```

def · defined in PropertyKindCalculus/
Specialization.lean

```

complete

```

def PropertyKindCalculus.MutuallyComparable
(E :
  PropertyKindCalculus.KindOfProperty →
  PropertyKindCalculus.KindOfProperty → Prop)
(a b : PropertyKindCalculus.KindOfProperty) : Prop

```

Proof for Definition 4.4.3

uses 0

An existential over a shared upper bound.

Theorem 4.4.4.

group

statement"uses 10

✓ L3

Owner Tags

Nicolas Rouquette

proved

Mutual comparability is reflexive (`refl`) and symmetric (`symm`), and any specialization induces it (`of_specializes`). Uses [mutual comparability](#).

Lean code for Theorem 4.4.4.

1 theorem

```

theorem · defined in PropertyKindCalculus/
Specialization.lean

```

complete

```

theorem PropertyKindCalculus.MutuallyComparable.symm
  {E :
    PropertyKindCalculus.KindOfProperty →
      PropertyKindCalculus.KindOfProperty → Prop}
  {a b : PropertyKindCalculus.KindOfProperty}
  (h : PropertyKindCalculus.MutuallyComparable E a b) :
    PropertyKindCalculus.MutuallyComparable E b a

```

Proof for Theorem 4.4.4

uses 0

Reflexivity takes the witness a itself; symmetry swaps the two halves of the conjunction.

4.5. Examination — principle, method, procedure (Dybæk Ch. 7, 14)

Definition 4.5.1.

group

uses 0

used by 4

✓ L3

The examination chain has three layers — an *examination principle* (§7.5 / VIM4 2CD 2.4), a *method* based on a principle (VIM4 2CD 2.5), and a *procedure* based on a method (VIM4 2CD 2.7) — gathered into one carrier `ExaminationItem` so that refinement is a homogeneous relation. The forgetful projection `basePrinciple` sends any item down to the principle it rests on.

Lean code for Definition 4.5.1.

1 definition

```

structure (1 field) · defined in PropertyKindCalculus/
                                Examination.lean

```

complete

```

structure PropertyKindCalculus.ExaminationPrinciple : Type

```

Proof for Definition 4.5.1

uses 0

Three small structures with `DecidableEq`, united by an `ExaminationItem` inductive; `basePrinciple` is the projection to the principle layer.

Definition 4.5.2.

group

statement" used by 2

✓ L3

Refinement `Refines` is the reflexive–transitive closure of the "based-on" edges (a procedure is based on its method, a method on its principle): `procedure` $\sqsubseteq$ `method` $\sqsubseteq$ `principle`. It mirrors [specialization](#) over kinds — the same closure construction, one level down at the defining aspect.

Lean code for Definition 4.5.2.

1 definition

```
inductive (2 constructors, Prop, 2 parameters) · defined in PropertyKindCalculus/Examination.lean
```

complete

```
inductive PropertyKindCalculus.Refines :
  PropertyKindCalculus.ExaminationItem →
    PropertyKindCalculus.ExaminationItem → Prop
```

Proof for Definition 4.5.2

uses 0

An inductive with refl and step constructors over a BasedOn edge relation; of_basedOn lifts a single edge.

Theorem 4.5.3.

group

statement" uses 10

✓ L3

Owner

Tags

Nicolas Rouquette

proved

Refinement is a preorder. It is reflexive (the refl constructor) and transitive (Refines.trans) on [the examination chain](#) — the same algebraic law proved for specialization, which an OWL2 reasoner can neither state nor prove.

Lean code for Theorem 4.5.3.

1 theorem

```
theorem · defined in PropertyKindCalculus/Examination.lean
```

complete

```
theorem PropertyKindCalculus.Refines.trans
  {a b c : PropertyKindCalculus.ExaminationItem}
  (h1 : PropertyKindCalculus.Refines a b)
  (h2 : PropertyKindCalculus.Refines b c) :
  PropertyKindCalculus.Refines a c
```

Proof for Theorem 4.5.3

uses 0

Transitivity is by induction on the first derivation, exactly as for Specializes: refl returns the second derivation, step re-applies the constructor under the inductive hypothesis.

Theorem 4.5.4.

group

statement" uses 10

✓ L3

Owner	Priority	Tags
Nicolas Rouquette	high	capstone proved

Refinement preserves the principle. Every examination in one refinement chain rests on the same base principle: $\mathrm{Refines} \mid a \mid b \mid ; \mathrm{Longrightarrow} ; \mathrm{basePrinciple} \mid a = \mathrm{basePrinciple} \mid b$. The forgetful projection to the principle layer is invariant along refinement — the examination-layer analogue of $\dim$ being a homomorphism, and one provable here in the Mathlib-free core without PhysLib. Builds on [refinement](#).

Lean code for Theorem 4.5.4.

1 theorem

theorem

· defined in

PropertyKindCalculus/
Examination.lean

complete

```

theorem PropertyKindCalculus.Refines.basePrinciple_eq
  {a b : PropertyKindCalculus.ExaminationItem}
  (h : PropertyKindCalculus.Refines a b) :
    a.basePrinciple = b.basePrinciple

```

Proof for Theorem 4.5.4

uses 0

By induction on the refinement derivation: each `Based0n` edge preserves the base principle (cases `<;>` `rfl`), and the inductive step chains the resulting equalities.

Theorem 4.5.5.

group

uses 2

used by 0

✓ L3

Owner	Tags
Nicolas Rouquette	proved

The examination principle is a defining aspect. Two [kinds](#) examined by different principles are distinct kinds — independently of scale and dimension. This is what keeps width and height distinct while both remain rational lengths, [mutually comparable](#) under their shared super-kind `Length`.

Lean code for Theorem 4.5.5.

1 theorem

theorem

· defined in

PropertyKindCalculus/
Examination.lean

complete

```

theorem PropertyKindCalculus.KindOfProperty.distinct_of_examPrinciple
  {k1 k2 : PropertyKindCalculus.KindOfProperty}
  (h : k1.examPrinciple ≠ k2.examPrinciple) : k1 ≠ k2

```

Proof for Theorem 4.5.5

uses 0

The kinds differ in their `examPrinciple` field, so they are unequal by congruence: substitute the assumed kind-equality and derive a contradiction from the field inequality.

4.6. Property value and value scale (Dybæk Ch. 9, 16, 10, 17)

Definition 4.6.1.

group

statement"used by 4

✓ L3

A *property value* (§9.15) bundles its [kind-of-property](#), a numeral (the numerical value, §16.10), and the reference the numeral is taken against — a metrological unit for a quantity value, a designation for a nominal one. A value *is a quantity value* (§16.10) iff its kind has magnitude, and two values are *comparable* iff they are of the same kind.

Lean code for Definition 4.6.1.

1 definition

```

structure (3 fields) · defined in PropertyKindCalculus/
  PropertyValue.lean

```

complete

```

structure PropertyKindCalculus.PropertyValue : Type

```

Proof for Definition 4.6.1

uses 0

A structure with `DecidableEq`, `scale`, `IsQuantityValue`, and `Comparable` are definitional projections over the kind field.

Theorem 4.6.2.

group

statement"used by 4

Owner

Tags

Nicolas Rouquette

proved

Comparability of values is an equivalence. Two [property values](#) are comparable iff they are of the same kind (§9.15); the relation is reflexive, symmetric, and transitive. Comparable values are moreover governed by the same scale type, so the same operators are defined on both (`scale_eq_of_comparable`). Transitivity is exactly the algebraic law a description logic cannot state.

Lean code for Theorem 4.6.2.

1 theorem

```
theorem · defined in PropertyKindCalculus/
PropertyKindCalculus/PropertyValue.lean

theorem PropertyKindCalculus.PropertyValue.Comparable.trans
  {u v w : PropertyKindCalculus.PropertyValue} (h1 : u.Comparable v)
  (h2 : v.Comparable w) : u.Comparable w
```

Proof for Theorem 4.6.2

uses 0

Comparability unfolds to equality of the kind field, so reflexivity, symmetry, and transitivity are those of Eq; scale_eq_of_comparable is congrArg on the scale projection.

Definition 4.6.3.

group

statement"statement"

A *property value scale* (§10.14) is the ordered set of possible, mutually comparable values of one [kind](#). It carries that kind and a provenance — *true* (consistent with the property's definition, §10.16.1) or *examined* (obtained by an examination procedure, §10.16.2). Its scaleType is the kind's scale, the datum Table 17.4 uses to gate which manipulations a scale admits, and KindOfProperty.valueScale gives the canonical true scale of a kind (§9.15 ↔ §10.14).

Lean code for Definition 4.6.3.

1 definition

```
structure (2 fields) · defined in PropertyKindCalculus/
PropertyKindCalculus/ValueScale.lean

structure PropertyKindCalculus.ValueScale : Type
```

Proof for Definition 4.6.3

uses 0

A structure carrying kind and a ScaleProvenance; Admits v is v.kind = s.kind, and KindOfProperty.valueScale builds the canonical true scale.

Theorem 4.6.4.

group

uses 3

used by 0

✓ L3

Owner	Priority	Tags
Nicolas Rouquette	high	capstone proved

A value scale holds only mutually comparable values. Any two values a [value scale](#) admits are of the scale's kind, hence [comparable](#) (§10.14) — the defining property of a value scale, quantified over its members. The scale type further licenses operations monotonically (allows_mono_of_le): a richer

value scale supports every manipulation a poorer one does (Table 17.4), inherited from [operator monotonicity](#).

Lean code for Theorem 4.6.4.

1 theorem

theorem · defined in PropertyKindCalculus/ValueScale.lean

complete

```
theorem PropertyKindCalculus.ValueScale.comparable_of_mem
  {s : PropertyKindCalculus.ValueScale}
  {v w : PropertyKindCalculus.PropertyValue} (hv : s.Admits v)
  (hw : s.Admits w) : v.Comparable w
```

Proof for Theorem 4.6.4

uses 0

Membership unfolds to equality with the scale's kind, so two members' kinds are equal by trans/symm; operator monotonicity is the scale-layer allows_mono re-exported through scaleType.

4.7. Metrological units (Dybæk Ch. 18, §13.3.3)

Definition 4.7.1.

group

uses 2

used by 12

✓ L3

A *metrological unit* (§18.12) carries the [kind](#) it references and a terminological symbol (the unit id, e.g. "m", "kg"). A kind *bears a unit* (BearsUnit, §13.3.3) iff its scale is at least differential, and a unit is *well-formed* iff its kind bears one; measure `u n` builds the [property value](#) "`n × symbol`".

Lean code for Definition 4.7.1.

1 definition

structure (2 fields) · defined in PropertyKindCalculus/Unit.lean

complete

```
structure PropertyKindCalculus.MetrologicalUnit : Type
```

Proof for Definition 4.7.1

uses 0

A structure with DecidableEq. BearsUnit is scale.AllowsDifference, WellFormed is kind.BearsUnit, and measure fills the value's numeral and reference fields from `n` and the unit's symbol.

Theorem 4.7.2.

group

statement"state

Owner

Tags

Nicolas Rouquette

proved

Commensurability is an equivalence. Two [units](#) are commensurable iff they reference the same kind (§9.13.4); the relation is reflexive, symmetric, and transitive. A metre and a centimetre are commensurable; a metre and a kilogram are not — incommensurability across kinds is a fact of the types, not a runtime check. Transitivity is exactly the law a description logic cannot state.

Lean code for Theorem 4.7.2.

1 theorem

theorem · defined in PropertyKindCalculus/
Unit.lean

complete

```
theorem PropertyKindCalculus.MetrologicalUnit.Commensurable.trans
  {u1 u2 u3 : PropertyKindCalculus.MetrologicalUnit}
  (h1 : u1.Commensurable u2) (h2 : u2.Commensurable u3) :
  u1.Commensurable u3
```

Proof for Theorem 4.7.2

uses 0

Commensurability unfolds to equality of the kind field, so reflexivity, symmetry, and transitivity are those of Eq.

Theorem 4.7.3.

group

statement" >us

Owner

Tags

Nicolas Rouquette

proved

Only a unitary kind bears a unit. A nominal kind has no magnitude, and — the distinctive §9.13.4 fact — an ordinal kind, though rankable, has no reference-times-number magnitude either, so neither bears a metrological unit (`not_wellFormed_of_nominal`, `not_wellFormed_of_ordinal`). Conversely a rational kind does (`rational_bears_unit`), and bearing a unit implies being a kind-of-quantity (`bearsUnit_isQuantity`). This gate is invisible to a dimension-and-scale-only view that treats every magnitude alike. Uses [the metrological unit](#).

Lean code for Theorem 4.7.3.

1 theorem

theorem · defined in PropertyKindCalculus/
Unit.lean

complete

```
theorem PropertyKindCalculus.MetrologicalUnit.not_wellFormed_of_ordinal
  {u : PropertyKindCalculus.MetrologicalUnit} (h : u.kind.IsOrdinal) :
  ¬u.WellFormed
```

Proof for Theorem 4.7.3

uses 0

BearsUnit is scale.AllowsDifference, which is False at nominal and ordinal; substitute the scale from IsNominal/IsOrdinal and the goal is $\neg$ False. The rational case and bearsUnit_isQuantity are the same four-way case split on the scale.

Theorem

4.7.4.

group

uses 3

used by 0

✓ L3

Owner	Priority	Tags
Nicolas Rouquette	high	capstone proved

The number-and-reference form is a faithful round-trip (§13.3.3). Measuring a quantity in a unit yields a numeral, and re-applying the unit to that numeral recovers the value: $\mathrm{numeral} \setminus, (\mathrm{measure} \setminus u \setminus n) = n \quad \text{and} \quad \mathrm{measure} \setminus u \setminus (\mathrm{numeral} \setminus v) = v$; $\text{when } v \text{ is measured in } u$. So quantity / unit = number and number $\times$ unit = quantity invert one another — the §13.3.3 "reference quantity multiplied by a number", checked as a law. Measuring in a well-formed unit moreover yields a [quantity value](#) (`measure_isQuantityValue`), and values measured in [commensurable](#) units are comparable. Builds on [the metrological unit](#).

Lean code for Theorem 4.7.4.

1 theorem

theorem · defined in PropertyKindCalculus/ Unit.lean

complete

```
theorem PropertyKindCalculus.MetrologicalUnit.measure_eq_of_measures
  {u : PropertyKindCalculus.MetrologicalUnit}
  {v : PropertyKindCalculus.PropertyValue} (h : u.Measures v) :
  u.measure v.numeral = v
```

Proof for Theorem 4.7.4

uses 0

`measure_numeral` is `rfl`. For the converse, destructure "measured in `u`" into the kind- and reference-equalities, unfold `measure`, and rewrite the unit's kind and `symbol` back to the value's fields; structure `eta` closes the goal.

5. Units and the Dimension-1 Problem

This chapter states the layer that originally motivated `PropertyKindCalculus`. Its motivating capstone — the dimension-1 disambiguation — is now *proved* (see the *Dimension as a Forgetful Functor* chapter for the forgetful map it rests on); the unit-arithmetic nodes around it remain *planned*, carrying informal

statements and proof sketches that appear as in-progress goals in the dependency graph until a (lean := ...) declaration or checked code block is attached.

PhysLib's `Dimension` (a $\mathbb{Q}$ -exponent free commutative group over the SI base quantities) makes every dimension-one quantity equal. Volumetric water content ($\mathrm{L^3/L^3}$), gravimetric water content ($\mathrm{M/M}$), relative permittivity, reflectivity, and emissivity are all dimension one — so a dimension-only type system cannot tell them apart. In soil-moisture retrieval that is precisely the class of confusion that silently corrupts a result. The kind layer is what restores the distinction.

5.1. Quantities and units (VIM4 2CD, Dybkær §13.3.3)

Definition

5.1.1.

group

uses 0

used by 4

✓ L3

A *numeric carrier* `Carrier R` is the minimal arithmetic a kind-indexed magnitude needs, supplied once per representation type `R` (a zero and an addition; a `LawfulCarrier` adds the additive-monoid laws). In plain engineering terms it is the *number type plug*: everything above it — kind discrimination, scale gating, the dimension functor, the interaction algebra, extensivity — is written against this class and so reused verbatim at every `R`. This is the role `TorchLean`'s `Context` α typeclass plays for tensor element types; the kind index is the layer *above* the carrier that `TorchLean` does not have.

Lean code for Definition 5.1.1.

1 definition

complete

class (2 methods) · defined in PropertyKindCalculus/Quantity.lean

```
class PropertyKindCalculus.Carrier (R : Type) : Type
```

Proof for Definition 5.1.1

uses 0

Realized as the `Carrier / LawfulCarrier` classes. `Carrier R` bundles zero and `add`; `LawfulCarrier R` extends it with associativity, commutativity, and the unit laws. `Int` and (in the `Dimension` library) $\mathbb{R}$ are lawful carriers; `Float` is a `Carrier` but deliberately *not* lawful (floating-point addition is not associative).

Definition

5.1.2.

group

uses 2

used by 7

✓ L3

A *quantity* `Quantity k R` is a magnitude of a fixed kind `k`, carried at a [representation type](#) `R` (`R10`). It is indexed by `k`, a [kind-of-property](#), so the type system forbids forming or comparing a `Quantity k1 R` with a `Quantity k2 R` when `k1 ≠ k2` — even if both kinds are dimension one. This indexing *is* the fix for the dimension-1 conflation. The second index `R` is the numeric carrier: the same value type is instantiated at $\mathbb{R}$ and `Int` to prove, at `Float` to run today, and (the planned float carriers) at `FP32` to bound rounding and at `IEEE32Exec` to run with NaN — the kind machinery above it written once for all of them.

Lean code for Definition 5.1.2.

1 definition

```
structure (1 field) · defined PropertyKindCalculus/
in Quantity.lean
```

complete

```
structure PropertyKindCalculus.Quantity (k :
PropertyKindCalculus.KindOfProperty)
(R : Type) : Type
```

Proof for Definition 5.1.2

uses 2

Realized as structure `Quantity (k : KindOfProperty) (R : Type)` carrying a magnitude in `R`, with same-kind addition `Quantity.add : Quantity k R → Quantity k R → Quantity k R` (`R4`: the type forces both summands to share the kind). The algebraic operations come from the [carrier](#) typeclass on `R`; which operators are even *admissible* is the orthogonal scale gate (the proved operator stratification, [operator monotonicity](#)).

Theorem 5.1.3.

group

uses 2

used by 2

✓ L3

Tags

Effort

medium

capstone

proved

Representation-parametric additivity (R10). The additivity laws on quantities — commutativity, associativity, and the zero unit — $q_1 + q_2 = q_2 + q_1$, $(q_1 + q_2) + q_3 = q_1 + (q_2 + q_3)$, $0 + q = q$ hold over every lawful carrier, established by a single proof and so available at each R at once: at $\mathbb{R}$ (the proof carrier) and at `Int` with no `R`-specific argument. The very carrier where these laws are absent — an executable `Float`, a [carrier](#) but not a lawful one — is what the `exec/` spec refinement bridge below exists to reconcile. Builds on [the quantity layer](#).

Lean code for Theorem 5.1.3.

1 theorem

```
theorem · defined PropertyKindCalculus/
in Quantity.lean
```

complete

```
theorem PropertyKindCalculus.Quantity.laws_parametric
{k : PropertyKindCalculus.KindOfProperty} {R : Type}
[PropertyKindCalculus.LawfulCarrier R]
(x y z : PropertyKindCalculus.Quantity k R) :
x.add y = y.add x ∧
(x.add y).add z = x.add (y.add z) ∧
```

```
PropertyKindCalculus.Quantity.zero.add x = x ∧
x.add PropertyKindCalculus.Quantity.zero = x
```

Proof for Theorem 5.1.3

uses 0

Proved in `PropertyKindCalculus.Quantity` (axiom-free). Each law unfolds `Quantity.add` to its carrier operation and rewrites by the corresponding `LawfulCarrier` field; `laws_parametric` bundles the four. Specializing to `Int` (core) and $\mathbb{R}$ (in the `Dimension` library) reuses the same proof verbatim, which is the R10 payoff.

Definition 5.1.4.

group

statement"statement"

A *carrier refinement* `CarrierRefinement E S` specifies how an exec carrier `E` (fast, lossy) stands in for a spec carrier `S` (exact): a forgetful map `toSpec : E → S` (the analogue of `TorchLean`'s `toReal`) and a spec-side round `: S → S`, with the single law that forgetting an exec addition equals rounding the spec addition, $\mathrm{toSpec}(x +_E y) = \mathrm{round}(\mathrm{toSpec} x +_S \mathrm{toSpec} y)$. In plain engineering terms it is the *rounding contract* between a slow exact number type and the fast machine number that stands in for it. Builds on the [carrier](#).

Lean code for Definition 5.1.4.

1 definition

class (4 methods) · defined in PropertyKindCalculus/QuantityRefinement.lean

complete

```
class PropertyKindCalculus.CarrierRefinement (E S : Type)
  [PropertyKindCalculus.Carrier E] [PropertyKindCalculus.Carrier S] :
  Type
```

Proof for Definition 5.1.4

uses 0

Realized as the `CarrierRefinement` class (core, axiom-free). `TorchLean`'s FP32 gives an *unconditional* instance over $\mathbb{R}$, and `IEEE32Exec` refines $\mathbb{R}$ on the finite path — both in the separately built `Torch` library.

Theorem 5.1.5.

group

uses 2

used by 0

✓ L3

Tags

Effort

Priority

large

high

capstone

proved

Exec refines spec across representations (R10). For a kind- k quantity, the exec sum viewed in the spec carrier is the rounding of the spec sum, $\mathrm{toSpec}(x +_E y) = \mathrm{round}(\mathrm{toSpec} x +_S \mathrm{toSpec} y)$, established over any [carrier refinement](#) and so holding at each at once. A law proved over the lawful spec carrier therefore transfers to the executable

run with one rounding step — the bridge that carries the [representation-parametric laws](#) from the lawful $\mathbb{R}$ onto a float that is not lawful, so one kind-indexed value serves both proof and execution.

Lean code for Theorem 5.1.5.

1 theorem

complete

theorem · defined in PropertyKindCalculus/
QuantityRefinement.lean

```
theorem PropertyKindCalculus.Quantity.add_refines
  {k : PropertyKindCalculus.KindOfProperty} {E S : Type}
  [PropertyKindCalculus.Carrier E] [PropertyKindCalculus.Carrier S]
  [PropertyKindCalculus.CarrierRefinement E S]
  (x y : PropertyKindCalculus.Quantity k E) :
  (x.add y).toSpec =
    PropertyKindCalculus.Quantity.roundBy
      (PropertyKindCalculus.CarrierRefinement.round E)
      (x.toSpec.add y.toSpec)
```

Proof for Theorem 5.1.5

uses 0

Proved as `Quantity.add_refines` (core, axiom-free): the `CarrierRefinement` bridge law lifted along the kind index. It is instantiated concretely in the separately built Torch library at TorchLean's `binary32` — an *unconditional* `CarrierRefinement FP32  $\mathbb{R}$` (the rounding spec has no overflow, so the law holds by computation) and a *conditional* `IEEE32Exec` refinement on the finite/no-overflow path (`Quantity.add_refines_exec`), where overflow is an explicit hypothesis rather than a silent failure. This is TorchLean's `BridgeFP32` pattern ("compute in $\mathbb{R}$, then round").

Definition 5.1.6.

group

uses 3

used by 0

✓ L3

Vector quantities: numerical array $\times$ one scalar unit (R11, ISO 80000-2 §18). A vector (or tensor) quantity is a *numerical* array carried at one kind with one *scalar unit* — not a per-coordinate collection of `number  $\times$  unit` values; "all units are scalars". This is the [carrier](#) taken at a function-space type: `Fin n  $\rightarrow$  R` is a (lawful) carrier whenever `R` is, pointwise, so a vector quantity is `Quantity k (Fin n  $\rightarrow$  R)` and the [additivity laws](#) transfer to it by the same parametric proof. The quantity is coordinate-independent; only its numerical components depend on the frame.

Lean code for Definition 5.1.6.

1 definition

complete

def · defined in PropertyKindCalculus/
QuantityVector.lean

```
def PropertyKindCalculus.instLawfulCarrierPi {ι R : Type}
  [PropertyKindCalculus.LawfulCarrier R] :
  PropertyKindCalculus.LawfulCarrier (ι → R)
```

Proof for Definition 5.1.6

uses 0

Realized as the pointwise `Carrier (ι → R) / LawfulCarrier (ι → R)` instances (core). The fuller structural apparatus — coordinate frames, tensor variance, transforms — remains a separate, owed axis.

Definition 5.1.7.

group

uses 2

statement">used by1 L3

A *unit* `Unit k` is a distinguished [quantity](#) of kind `k` chosen as the reference: measuring expresses any quantity of kind `k` as `\text{number} \times \text{unit}`. Because `Unit` is indexed by the same `k`, "a metre" and "a unit of gravimetric water content" inhabit different types. This is the real-valued refinement of the proved [metrological unit](#): it replaces the symbolic numeral-and-reference form with a magnitude in $\mathbb{R}$, so that conversion *ratios* become arithmetic.

Proof for Definition 5.1.7

uses 0

Planned. A `Unit k := { ref : Quantity k }` (or a chosen nonzero reference), with `measure : Quantity k → Unit k → ℝ` giving the numeric value in that unit.

Theorem 5.1.8.

group

statement">us

Effort	Priority	Tags
medium	high	capstone
		planned

Unit conversion is a faithful round-trip. For two units u_1, u_2 of the same kind k , converting a quantity from u_1 to u_2 and back is the identity: $\mathrm{convert}_{u_2 \rightarrow u_1}(\mathrm{convert}_{u_1 \rightarrow u_2}(q)) = q$. Conversion is multiplication by the ratio u_1 / u_2 , defined only within a kind. There is no conversion between units of different kinds — that is a type error, not a runtime check. Builds on [the unit definition](#).

Proof for Theorem 5.1.8

uses 0

Planned. The conversion factor is $r = \mathrm{measure}(u_1, u_2)$ and its inverse r^{-1} ; the round-trip is $r^{-1} \cdot (r \cdot q) = q$, requiring $r \neq 0$ from the chosen-reference nonzeroness. Ratio formation is licensed by the ratio scale.

Theorem 5.1.9.

group

uses 2

statement">used by1 L3

Effort	Priority
small	high

Tags

capstone

proved

The dimension-1 disambiguation (the motivating capstone). There exist distinct kinds that map to the same dimension. Concretely, for volumetric and gravimetric water content, $\mathrm{vwc} \neq \mathrm{gwc} \sqquad \text{yet} \sqquad \dim(\mathrm{vwc}) = \dim(\mathrm{gwc}) = \mathbf{1}$, and likewise for permittivity and reflectivity. So the kind layer is a strict refinement of the dimension layer: $\dim$ is not injective. This is the statement *PhysLib* — and any dimension-only model, whether OWL2 or a representation-rooted hierarchy like SysML v2's — structurally cannot make. Uses [kind-of-property](#) and [the dimension map](#).

Lean code for Theorem 5.1.9.

1 theorem

theorem · defined in PropertyKindCalculus/
Dimension.lean

complete

```
theorem PropertyKindCalculus.dim_not_injective :
  ∃ a b,
    a.kind ≠ b.kind ∧ a.toDimension = b.toDimension ∧ a.toDimension = 1
```

Proof for Theorem 5.1.9

uses 0

Proved in the Dimension library. `vwc` and `gwc` are declared as dimensioned kinds with distinct kind ids, so `vwc.kind ≠ gwc.kind` is by `decide`; both carry `dim = 1`, so their dimensions agree by `rf1`. Together they witness $\exists a\ b, a.\mathrm{kind} \neq b.\mathrm{kind} \wedge \dim a = \dim b \wedge \dim a = 1$.

5.2. Verified classification and instantiable kind-laws (R12)

A quantity `Quantity k R` records its kind `k` as a *tag*: the magnitude is an arbitrary `R`, so a value can be labelled with a kind it has not earned (⊗ type-checks as an area). R12 upgrades a classification from an *assertion* to a *certificate*: a quantity is classified under a kind by a proof that it satisfies the kind's *defining relation* — the formalized ISO 80000 *Remark* — to quantities of the kinds it is built from. The kind becomes a *refinement* (a tag plus its earned proof), not a label.

The payoff is *instantiation*. Every kind-law is stated as a carrier-parametric universal over quantities with the kind-relation as a premise; "instantiating it at the quantity level" is then ordinary application, and any property of a kind's defining relation transports to every quantity certified under it. This is the inter-kind generalization of the carrier-parametric laws of R10.

Definition 5.2.1.

group

uses 0

statement">used by 1 L

A *product kind-law* `ProductKind k1 k2 k` records that `k` is the product kind of `k1` and `k2`. In the core it carries the scale precondition — all three are ratio-scale, since only ratio quantities multiply (Dybæk §13.3.5); the dimension layer refines it with the dimensional equation $k.\text{dim} = k_1.\text{dim} \cdot k_2.\text{dim}$.

Lean code for Definition 5.2.1.

1 definition

```
structure (3 fields) · defined in PropertyKindCalculus/
QuantityClassification.lean
```

complete

```
structure PropertyKindCalculus.ProductKind
(k1 k2 k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 5.2.1

uses 0

Realized as structure `ProductKind` with three `IsRational` fields. A concrete law such as `area = length · length` is the value `(rfl, rfl, rfl)`.

Definition 5.2.2.

group

statement" uses 10

✓ L

The *certificate* `q.IsProduct h a b` states that `q` (of kind `k`) is the product of `a` (of `k1`) and `b` (of `k2`): its magnitude is the product of theirs. A proof of this certifies `q`'s classification, rather than asserting it. It is a separate proposition, so `Quantity` stays a clean tag and certificates are carried only when needed. The smart constructor `Quantity.mul` yields a quantity whose certificate holds by construction.

Lean code for Definition 5.2.2.

1 definition

```
def · defined in PropertyKindCalculus/
QuantityClassification.lean
```

complete

```
def PropertyKindCalculus.Quantity.IsProduct {R : Type} [Mul R]
{k1 k2 k : PropertyKindCalculus.KindOfProperty}
(_h : PropertyKindCalculus.ProductKind k1 k2 k)
(q : PropertyKindCalculus.Quantity k R)
(a : PropertyKindCalculus.Quantity k1 R)
(b : PropertyKindCalculus.Quantity k2 R) : Prop
```

Proof for Definition 5.2.2

uses 0

`Quantity.IsProduct h q a b := q.magnitude = a.magnitude * b.magnitude` over a carrier with multiplication; `Quantity.mul h a b := {a.magnitude * b.magnitude}`, and `mul_isProduct` is `rfl`.

Theorem 5.2.3.

group

uses 0

used by 0

✓ L

Effort

small

Tags

proved

A kind-law that instantiates at the quantity level. A quantity certified as the product of a and b is unique — and any certified product equals the smart-constructed one (`eq_mul_of_isProduct`, `canonicity`). Supplying concrete quantities gives the quantity-level fact by application. Axiom-free.

Lean code for Theorem 5.2.3.

1 theorem

theorem · defined in PropertyKindCalculus/
QuantityClassification.lean

complete

```
theorem PropertyKindCalculus.Quantity.isProduct_unique {R : Type} [Mul R]
  {k1 k2 k : PropertyKindCalculus.KindOfProperty}
  (h : PropertyKindCalculus.ProductKind k1 k2 k)
  {q q' : PropertyKindCalculus.Quantity k R}
  {a : PropertyKindCalculus.Quantity k1 R}
  {b : PropertyKindCalculus.Quantity k2 R}
  (hq : PropertyKindCalculus.Quantity.IsProduct h q a b)
  (hq' : PropertyKindCalculus.Quantity.IsProduct h q' a b) : q = q'
```

Proof for Theorem 5.2.3

uses 0

`isProduct_unique` rewrites both magnitudes to `a.magnitude * b.magnitude` and closes by structure `η`; `eq_mul_of_isProduct` is `isProduct_unique` against `mul_isProduct`. Both depend on no axioms.

Definition 5.2.4.

group

uses 0

used by 0

✓ L3

A *quotient kind-law* `QuotientKind k1 k2 k` records that k is the quotient kind $k = k_1 / k_2$ (speed = length / duration; plane angle = arc / radius, where the lengths cancel). Like the product, it carries the ratio-scale precondition in the core and is refined by $k.\text{dim} = k_1.\text{dim} / k_2.\text{dim}$ in the dimension layer. Its certificate `Quantity.IsQuotient` and smart constructor `Quantity.div` mirror the product family exactly, over the core `Div` class.

Lean code for Definition 5.2.4.

1 definition

structure (3 fields) · defined in PropertyKindCalculus/
QuantityClassification.lean

complete

```
structure PropertyKindCalculus.QuotientKind
  (k1 k2 k : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 5.2.4

uses 0

structure QuotientKind with three IsRational fields; Quantity.div h a b := {a.magnitude / b.magnitude}, div_isQuotient is rfl, and isQuotient_unique / eq_div_of_isQuotient are the product family's proofs transcribed. Axiom-free.

Definition 5.2.5.

group

uses 0

statement">used by 1 L

A *reciprocal kind-law* ReciprocalKind k_1 k records that $k = 1 / k_1$ (frequency = 1 / period; curvature = 1 / radius; repetency = 1 / wavelength) — the unary special case, over the core Inv class, refined by $k.\text{dim} = (k_1.\text{dim})^{-1}$. All three families (product, quotient, reciprocal) stay Mathlib-free, using only toolchain-level arithmetic classes.

Lean code for Definition 5.2.5.

1 definition

```
structure (2 fields) · defined in PropertyKindCalculus/  
QuantityClassification.lean
```

complete

```
structure PropertyKindCalculus.ReciprocalKind  
  ( $k_1$   $k$  : PropertyKindCalculus.KindOfProperty) : Prop
```

Proof for Definition 5.2.5

uses 0

structure ReciprocalKind with two IsRational fields; Quantity.recip h a := {a.magnitude⁻¹}, recip_isReciprocal is rfl, with isReciprocal_unique / eq_recip_of_isReciprocal as before. Axiom-free.

6. Dimension as a Forgetful Functor

PropertyKindCalculus does not discard PhysLib's Dimension; it keeps it as a forgetful functor. Dimension answers the coarse question — "are these even commensurable in the SI base quantities?" — and the kind layer answers the fine one. The forgetful map, its multiplicativity, and the dimension-1 disambiguation it makes possible are now *realized*, sorry-free, in the Dimension library; the full coherence law — that $\backslash\text{dim}$ is a homomorphism over the interaction algebra — is realized in the Interaction module and stated as the homomorphism capstone below.

Two terms recur below; in plain engineering terms they mean this. A *forgetful functor* is a deliberately lossy, one-way map: $\backslash\text{dim}$ keeps only a kind's SI base-quantity exponents (length, mass, time, ...) and discards everything else that distinguishes it, so many kinds share one dimension and there is no inverse. A *homomorphism* is a map that respects an operation: here $\backslash\text{dim}$ respects multiplication — the dimension of a product is the product of the dimensions, the exponents adding — which is exactly the bookkeeping of everyday dimensional analysis, stated and proved here as a law rather than performed by hand.

6.1. The forgetful map (Dybckær Ch. 19)

Definition 6.1.1.

group

uses 2

used by 13

✓ L3

The *dimension map* $\backslash\text{dim}$ sends a *dimensioned kind* — a [kind-of-property](#) paired with a PhysLib Dimension ($\mathbb{Q}$ -exponents over length/time/mass/charge/temperature) — to that dimension. It is deliberately many-to-one: that is what makes [the dimension-1 disambiguation](#) true.

Lean code for Definition 6.1.1.

1 definition

```
def · defined in PropertyKindCalculus/  
Dimension.lean
```

complete

```
def PropertyKindCalculus.DimensionedKind.toDimension  
(dk : PropertyKindCalculus.DimensionedKind) : Dimension
```

Proof for Definition 6.1.1

uses 0

Realized in the Dimension library (dimension/). DimensionedKind is the structure pairing a kind with its Dimension; toDimension is the forgetful projection. PhysLib + Mathlib enter only here, as a separate library; the spine stays Mathlib-free.

Theorem 6.1.2.

group

uses 2

statement">used by 1 L3

Tags

proved

The forgetful functor preserves products. The dimension of a product of dimensioned kinds is the product of their dimensions, $\backslash\text{dim } (a \backslash\text{cdot } b) = \backslash\text{dim } a \backslash\text{cdot } \backslash\text{dim } b$, the SI exponents adding — ordinary dimensional bookkeeping, proved as a law. This is the dimension-component of functoriality, stated for the provisional product; the full interaction-algebra version is [the homomorphism capstone](#) below. Builds on [the dimension map](#).

Lean code for Theorem 6.1.2.

1 theorem

```
theorem · defined in PropertyKindCalculus/  
Dimension.lean
```

complete

```
theorem PropertyKindCalculus.DimensionedKind.toDimension_times  
(a b : PropertyKindCalculus.DimensionedKind) :  
(a.times b).toDimension = a.toDimension * b.toDimension
```

Proof for Theorem 6.1.2

uses 0

The provisional product multiplies the dimension components (PhysLib's `Dimension.mul`, i.e. exponent addition), so the equation holds by `rfl`. The unit law $\dim 1 = 1$ holds likewise.

Theorem

6.1.3.

group

uses 2

used by 3

✓ L

Tags

Effort

medium

Priority

high

capstone

proved

Dimensional coherence — $\dim$ is a homomorphism over the full interaction algebra. Whenever the interaction algebra says k_1 and k_2 combine to k_3 , their dimensions combine the same way: $\mathrm{KMul}\ k_1\ k_2\ k_3$; $\Rightarrow \dim k_3 = \dim k_1 \cdot \dim k_2$. This is the theorem that licenses forgetting to the dimension layer without losing soundness: a kind-level product is always dimensionally consistent. It upgrades [the proved multiplicativity](#) from the provisional product to Flater's gated [kind product](#).

Lean code for Theorem 6.1.3.

1 theorem

theorem · defined in PropertyKindCalculus/Interaction.lean

complete

```
theorem PropertyKindCalculus.InteractionAlgebra.dim_homomorphism
  (A : PropertyKindCalculus.InteractionAlgebra)
  {a b c : PropertyKindCalculus.DimensionedKind} (h : A.KMul a b c) :
  c.toDimension = a.toDimension * b.toDimension
```

Proof for Theorem 6.1.3

uses 0

Realized in the Interaction module as `InteractionAlgebra.dim_homomorphism`. Coherence is a well-formedness field of every `InteractionAlgebra`, so the homomorphism is its projection: any algebra that can be formed at all already guarantees $\dim k_3 = \dim k_1 \cdot \dim k_2$ on every sanctioned edge. The content sits in the worked algebra, where the obligation is discharged by the `Dimension-group` computation, case by case.

7. The Interaction Algebra (Flater Appendix C)

This is the contribution beyond the VIM4 2CD and OML, and the reason Flater's note is titled *full tracking of kinds of quantities*. Dimensional analysis says torque and energy share a dimension ($\mathrm{M}\backslash, L^2\backslash, T^{-2}\backslash$), yet they are not the same kind and not interchangeable. The interaction algebra records which kinds legitimately combine, with the dimensional product as a coherence side-

condition rather than the definition. It is now *realized*, sorry-free, in the Interaction module: the curated product, its division dual, the multiplication–division round-trip, and the worked torque-versus-energy instance.

7.1. The ternary product of kinds (Flater App. C)

Definition 7.1.1. group uses 2 used by 4 ✓ L3
 $\mathrm{KMul} \setminus k_1 \setminus k_2 \setminus k_3$ holds when kind k_1 times kind k_2 yields kind k_3 . It is partial and curated: an application asserts the legitimate products of its [kinds](#), each carrying the dimensional coherence obligation discharged by [the homomorphism theorem](#).

Lean code for Definition 7.1.1.

1 definition

abbrev · defined in PropertyKindCalculus/Interaction.lean complete

abbrev PropertyKindCalculus.InteractionAlgebra.KMul
 (*self* : PropertyKindCalculus.InteractionAlgebra) :
 PropertyKindCalculus.DimensionedKind →
 PropertyKindCalculus.DimensionedKind →
 PropertyKindCalculus.DimensionedKind → Prop

Proof for Definition 7.1.1 proof" >uses 1
Realized as the KMul field of `InteractionAlgebra`: a ternary relation over [dimensioned kinds](#), paired in the same structure with the well-formedness field `kMul_coherent` that requires every edge to satisfy $\dim k_3 = \dim k_1 \cdot \dim k_2$. The structure cannot be formed for an algebra that violates coherence.

Definition 7.1.2. group statement" >statement" >used by 1 L3
 $\mathrm{KDiv} \setminus k_3 \setminus k_2 \setminus k_1$ is the division dual of [the kind product](#): k_3 divided by k_2 yields k_1 . Fuel-consumption $\times$ rainfall is the classic Flater example whose careless handling yields a category error rather than a number.

Lean code for Definition 7.1.2.

1 definition

def · defined in PropertyKindCalculus/Interaction.lean complete

```
def PropertyKindCalculus.InteractionAlgebra.KDiv
  (A : PropertyKindCalculus.InteractionAlgebra)
  (c b a : PropertyKindCalculus.DimensionedKind) : Prop
```

Proof for Definition 7.1.2

uses 0

Realized as `InteractionAlgebra.KDiv`, defined as the product read backwards ($\mathrm{KDiv} \backslash c \backslash b \backslash a := \mathrm{KMul} \backslash a \backslash b \backslash c$), so it inherits the same partiality and coherence side-condition.

Theorem

7.1.3.

group

uses 3

used by 0

✓ L3

Effort	Priority	Tags
medium	high	capstone
		proved

Multiplication and division are inverse on kinds. A product and its quotient hold together — the round-trip is an equivalence: $\mathrm{KMul} \backslash k_1 \backslash k_2 \backslash k_3 \vdash \mathrm{KDiv} \backslash k_3 \backslash k_2 \backslash k_1$. Together with [dimensional coherence](#) this is the calculus that OWL2 cannot host: an algebraic law with arithmetic side-conditions, proved rather than consistency-checked. Builds on [the kind product](#) and [the kind quotient](#).

Lean code for Theorem 7.1.3.

1 theorem

theorem · defined in PropertyKindCalculus/Interaction.lean

complete

```
theorem PropertyKindCalculus.InteractionAlgebra.kMul_iff_kDiv
  (A : PropertyKindCalculus.InteractionAlgebra)
  (a b c : PropertyKindCalculus.DimensionedKind) :
  A.KMul a b c ↔ A.KDiv c b a
```

Proof for Theorem 7.1.3

uses 0

Immediate from defining `KDiv` as the converse of `KMul`: the biconditional is `Iff.rfl`. The round-trip is the symmetry of that definition.

Theorem

7.1.4.

group

statement" uses 1

✓ L3

Effort	Tags
small	proved

Worked instance. Torque times plane angle is energy, and energy is not torque, even though $\dim(\mathrm{torque}) = \dim(\mathrm{energy})$. Formally $\mathrm{KMul} \setminus \mathrm{torque} \setminus \mathrm{angle} \setminus \mathrm{energy}$ holds while $\mathrm{energy} \neq \mathrm{torque}$ — the small example that shows the algebra carrying information dimension alone discards. Uses [the kind product](#).

Lean code for Theorem 7.1.4.

1 theorem

theorem · defined in PropertyKindCalculus/Interaction.lean

complete

```
theorem PropertyKindCalculus.torque_angle_work :
  PropertyKindCalculus.siMech.KMul PropertyKindCalculus.torqueKind
    PropertyKindCalculus.angleKind PropertyKindCalculus.energyKind ∧
  PropertyKindCalculus.energyKind.kind ≠
    PropertyKindCalculus.torqueKind.kind ∧
  PropertyKindCalculus.energyKind.toDimension =
    PropertyKindCalculus.torqueKind.toDimension
```

Proof for Theorem 7.1.4

uses 0

Proved in the Interaction module. torque, angle (dimension one), and energy are declared as dimensioned kinds, with torque and energy both given dimension force · length; the product edge torque_angle_energy is asserted in the worked siMech algebra and its coherence checked by the Dimension-group computation, while energy.kind ≠ torque.kind is by decide.

8. Extensivity

Dybæk §13.5 distinguishes *extensive* kinds (whose value over a whole is the sum over disjoint parts) from kinds where that fails. This is a quantified arithmetic law over the mereology of systems — again something a description logic cannot state. It is now *realized*, sorry-free, in the Mathlib-free Extensivity module: the decomposition mereology, the Extensive predicate, the induction that lifts a single split to the whole tree, and the volume-on-mixing counterexample.

8.1. Extensive and conditionally-extensive kinds (Dybæk §13.5)

Definition 8.1.1.

group

statement"uses by 2

✓ L

A kind is *extensive* when, for any partition of a [system](#) into disjoint parts, the quantity of the whole equals the sum over the parts. This requires the mereological structure deferred from the foundations chapter.

Lean code for Definition 8.1.1.

1 definition

```
structure (2 fields) · defined in PropertyKindCalculus/
                                Extensivity.lean

structure PropertyKindCalculus.Extensive (k :
PropertyKindCalculus.KindOfProperty)
(m : PropertyKindCalculus.Measurement) : Prop
```

Proof for Definition 8.1.1

uses 0

Realized as `Extensive k m`, a predicate over a kind `k` and a measurement `m` (the property value observed on each node of a `Decomposition`): it asserts that every part is measured as a value of kind `k`, and that the numeral on a disjoint union is the sum of the numerals on its two parts.

Theorem

8.1.2.

group

statement" uses 10

✓ L3

Effort
medium

Tags
capstone
proved

Extensive aggregation. For an extensive kind k and a system decomposed into parts $s_1, \dots, s_n$, $\mathrm{value}_k(\bigsqcup_i s_i) = \sum_i \mathrm{value}_k(s_i)$. The *forall-quantified additivity law itself* — not a single instance — is the deliverable, and it is exactly what OWL2 has no vocabulary for. Builds on [extensive kinds](#).

Lean code for Theorem 8.1.2.

1 theorem

```
theorem · defined in PropertyKindCalculus/
                    Extensivity.lean

theorem PropertyKindCalculus.extensive_additive
{ k : PropertyKindCalculus.KindOfProperty }
{ m : PropertyKindCalculus.Measurement }
(h : PropertyKindCalculus.Extensive k m)
(d : PropertyKindCalculus.Decomposition) :
(m d).numeral = PropertyKindCalculus.leafSum m d
```

Proof for Theorem 8.1.2

uses 0

Proved in the Extensivity module by induction on the Decomposition: at a leaf the law is reflexivity; at a union the single-split additive field plus the two induction hypotheses give the numeral of the whole as the sum of the leaf sums.

Theorem 8.1.3. group statement uses 0

Effort

small

Tags

proved

Counterexample — volume on mixing is not extensive. Volume fails the additivity law for a water/ethanol mixture: the volume of the mixture is strictly less than the sum of the component volumes. Stating the negation keeps the extensive predicate honest. Uses [extensive kinds](#).

Lean code for Theorem 8.1.3.

1 theorem

theorem · defined in PropertyKindCalculus/Extensivity.lean

complete

```
theorem PropertyKindCalculus.mixing_subadditive :
  (PropertyKindCalculus.volMix PropertyKindCalculus.mixture).numeral <
    (PropertyKindCalculus.volMix
      PropertyKindCalculus.waterPart).numeral +
      (PropertyKindCalculus.volMix
        PropertyKindCalculus.ethanolPart).numeral ∧
    ¬PropertyKindCalculus.Extensive PropertyKindCalculus.volume
      PropertyKindCalculus.volMix
```

Proof for Theorem 8.1.3

uses 0

Proved in the Extensivity module. 50 mL of water and 50 mL of ethanol measure 96 mL when mixed; the strict inequality $96 < 50 + 50$ is by `decide`, and the same `volMix` measurement witnesses $\neg \text{Extensive volume}$ by refuting its additive field on that two-part decomposition.

9. ISO/IEC 80000 — The Standards Catalogue

The quantity-kinds and units specified here are grounded against the international standard series ISO/IEC 80000, *Quantities and units*. That series is **licensed and copyrighted**: no normative content from it is reproduced in this work. What is recorded is the *citable identity* of each part — body, number, part, edition, and publication date — so that every catalogued quantity-kind or unit can name its exact source. This chapter specifies that references layer; the chapters that follow specify one part's quantity-kinds and units at a time, each added as coverage of that part becomes available.

9.1. The references layer (citation metadata only)

Definition 9.1.1.

group

uses 0

statement

A *standard reference* `StandardRef` is the bibliographic identity of one part: its publishing body (ISO or IEC), the series number (80000), the part number, the edition designation as printed, the publication date, and the part's English title. It carries **no** normative content — only what is needed to *cite* the source. Its `cite` renders the conventional form, e.g. IEC 80000-6, Edition 2.0, 2022-11.

Lean code for Definition 9.1.1.

1 definition

structure (6 fields) · defined in `PropertyKindCalculus/Iso80000/References.lean`

complete

```
structure PropertyKindCalculus.Iso80000.StandardRef : Type
```

Proof for Definition 9.1.1

uses 0

Realized as structure `StandardRef` (Mathlib-free) with a `StandardBody` (ISO/IEC) field and the printed edition, date, and title as strings, plus designation (ISO 80000-3) and `cite` (the full reference). The values are read off each licensed part's cover page; nothing normative is transcribed.

Definition 9.1.2.

group

statement" >

The *catalogue* is the list of the twelve catalogued parts in part order, each a [standard reference](#). It is the single place this work names *which editions* of ISO/IEC 80000 the catalogued quantity-kinds and units are grounded against, so the grounding is auditable rather than implicit.

Lean code for Definition 9.1.2.

1 definition

def · defined in `PropertyKindCalculus/Iso80000/References.lean`

complete

```
def PropertyKindCalculus.Iso80000.catalogue :  
  List PropertyKindCalculus.Iso80000.StandardRef
```

Proof for Definition 9.1.2

uses 0

Realized as `catalogue : List StandardRef`, the twelve definitions `iso80000_1 ... iso80000_12` (with `iec80000_6` the one IEC-published part). The edition is "Second edition" for every part except part 6's "Edition 2.0"; the dates are the cover dates of the licensed copies.

10. ISO 80000-3 — Space and Time

The first part whose quantity-kinds and units are specified in full is ISO 80000-3, *Space and time*. Every item — all of 3-1.1 ... 3-26.3 — is catalogued: length, the width/height/distance/radius family, area, volume, the angles, duration, the frequencies, velocity and speed, acceleration, the wave quantities, and the rest, each carrying its exact source as data: the part, the printed item designation, the principal quantity symbol, and the coherent SI unit symbol. Only **citation locators** are recorded; no normative content (definitions, remarks) from the licensed standard is reproduced. The defining *mathematics* of selected remarks is formalized in its own right, in the sibling modules linked below.

Part 3 turns out to be an unusually good proving ground for the calculus, because the standard itself is full of distinctions that a dimension-only or representation-only model cannot make: a dozen different length quantities, several dimensionless angles, and three families of quantities that share a dimension while remaining distinct kinds.

10.1. Quantity-kinds and units of ISO 80000-3

Definition10.1.1.

groupstatement"uses"10

A *catalogued kind* pairs a dimensioned [quantity](#)-kind with its exact source in the series — the part, the printed item designation (e.g. "3-3"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream rather than kept in a comment.

Lean code for Definition10.1.1.

1 definition

structure (5 fields) · defined inPropertyKindCalculus/Iso80000/Part3.lean

complete

structure PropertyKindCalculus.Iso80000.Part3.CataloguedKind : Type

Proof for Definition 10.1.1

uses"0

Realized as structure CataloguedKind over a DimensionedKind, with cite rendering e.g. ISO 80000-3, Second edition, 2019-10 item 3-3. The catalogue lists all 42 items in item order — length (3-1.1), area (3-3), volume (3-4), duration (3-9), speed (3-10.2), frequency (3-17.1), ... — each paired with its coherent SI unit symbol.

Theorem10.1.2.

groupstatement"uses"10

EffortTags

small

proved

Area is L^2 (item 3-3). The catalogued area kind carries length-exponent two in PhysLib's dimension group — the standard's dimensional statement reproduced as a checked computation, not an annotation. Uses [the dimension map](#).

Lean code for Theorem 10.1.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.area_dim_length :
  PropertyKindCalculus.Iso80000.Part3.area.dim.length = 2
```

Proof for Theorem 10.1.2

uses 0

Proved as area_dim_length : area.dim.length = 2 in the Iso80000 library, discharged by Dim.area_length from the Dimension layer. The companion volume_dim_length (item 3-4) gives length-exponent three.

Theorem 10.1.3.

group

statement" uses 1 0

✓ L

Effort

Tags

small

proved

Speed is $L \cdot T^{-1}$ (item 3-10.2). The catalogued speed kind's dimension equals length divided by time — a checked computation in the dimension group. Uses [the dimension map](#).

Lean code for Theorem 10.1.3.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.speed_dim :
  PropertyKindCalculus.Iso80000.Part3.speed.dim =
    PropertyKindCalculus.Dim.length / PropertyKindCalculus.Dim.time
```

Proof for Theorem 10.1.3

uses 0

Proved as speed_dim : speed.dim = Dim.length / Dim.time, discharged by Dim.speed_eq. The catalogue also records curvature_dim_length = -1 (item 3-2, L^{-1}), frequency_dim_time = -1 (item 3-17.1, T^{-1}), and acceleration_dim_time = -2 (item 3-11, $L \cdot T^{-2}$).

Theorem 10.1.4.

group

statement" uses 1 0

✓ L

Effort

small

Tags

proved

The metre is a well-formed unit (item 3-1.1). Length is a ratio-scale kind, so it bears a [unit](#) (Dybkaer §13.3.5); the metre is one such chosen reference.

Lean code for Theorem 10.1.4.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.metre_wellFormed :
  PropertyKindCalculus.Iso80000.Part3.metre.WellFormed
```

Proof for Theorem 10.1.4

uses 0

Proved as `metre_wellFormed : metre.WellFormed` via `KindOfProperty.rational_bears_unit`.

Theorem 10.1.5.

group

statement"uses 10

✓ L3

Effort

small

Tags

proved

A metre and a second are not commensurable. Length and duration are distinct kinds, so their units cannot be compared by ratio — a type-level fact, not a runtime check. Uses [metrological units](#).

Lean code for Theorem 10.1.5.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.metre_second_not_commensurable
  ¬PropertyKindCalculus.Iso80000.Part3.metre.Commensurable
  PropertyKindCalculus.Iso80000.Part3.second
```

Proof for Theorem 10.1.5

uses 0

Proved as `metre_second_not_commensurable : ¬ metre.Commensurable second` by unfolding `Commensurable` to the underlying kinds and `decide` (length's kind id differs from duration's).

10.2. The length family: distinguishing kinds by measurement principle

ISO 80000-3 lists width (3-1.2), height (3-1.3), thickness (3-1.4), diameter (3-1.5), radius (3-1.6), path length (3-1.7), distance (3-1.8), and more as *separate items* — yet every one of them has dimension L and ratio scale. The standard distinguishes them only in *prose*: a width is a transverse extent, a distance is a shortest path, and so on. A dimension-only or representation-only model cannot capture the difference at all; it sees one type, "a real number of metres", for all of them.

This is exactly requirement *R2*, and Part 3 is where it earns its keep on the real standard. Item 3-1.1, *length*, is the broad genus. Each later item is specified as a *species* of length, distinguished **not by fiat** (not merely by a different name) but by an explicit *measurement principle* — Dybkær's examination principle (§7.5), a genuine defining aspect. The species are then provably distinct kinds, while remaining *mutually comparable* as lengths; and the specialization edges mirror the standard's own "defined in terms of" links (a diameter is a width of a circle, a radius is half a diameter, a distance is a shortest path length).

Definition 10.2.1.

group

uses 2

used by 0

✓ L3

A *length species* is a ratio-scale [quantity](#)-kind of dimension L carrying an [examination principle](#) — this work's own terse descriptor of the measurement principle that distinguishes it (transverse extent, shortest path, centre-to-rim, ...), never the standard's normative definition. Width, height, thickness, diameter, radius, path length, distance, radial distance, the position and displacement vectors, the radius of curvature, and the wavelength are all species of the general length kind (item 3-1.1).

Lean code for Definition 10.2.1.

1 definition

```
def · defined in PropertyKindCalculus/Iso80000/Part3.lean
```

complete

```
def PropertyKindCalculus.Iso80000.Part3.lengthSpecies (id : String)
  (p : PropertyKindCalculus.ExaminationPrinciple) :
  PropertyKindCalculus.DimensionedKind
```

Proof for Definition 10.2.1

uses 0

lengthSpecies id p := { kind := { id, scale := .ratio, examPrinciple := some p.id }, dim := Dim.length }. The general length carries no examination principle; the direct-parent edges of the family are collected in an Edge relation mirroring the standard's definitional links (thickness and diameter via width, radius via diameter, distance via path length, ...).

Theorem 10.2.2.

group

statement" uses 1 0

✓ L3

Effort

small

Tags

proved

Width and distance are distinct kinds — by measurement principle, not by fiat. A width (transverse extent) and a distance (shortest path) are different kinds because they are examined by different principles, proved through `distinct_of_examPrinciple` (§7.5) rather than by appealing to their `id` strings. Same dimension `L`, same scale, different kind. Uses [the examination principle](#).

Lean code for Theorem 10.2.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.width_ne_distance :
  PropertyKindCalculus.Iso80000.Part3.width.kind ≠
    PropertyKindCalculus.Iso80000.Part3.distance.kind
```

Proof for Theorem 10.2.2

uses 0

`width_ne_distance` : `width.kind` $\neq$ `distance.kind`, from `KindOfProperty.distinct_of_examPrinciple` applied to the differing examination principles `transverse-extent` and `shortest-path` (a decide on the `Option String` links). The companion `width_examinedBy` checks the kind-to-principle link. Axiom-free.

Theorem 10.2.3.

group

statement"uses 10

✓ L3

Effort

small

Tags

proved

A radius specializes length — transitively. Through the chain `radius` $\sqsubseteq$ `diameter` $\sqsubseteq$ `width` $\sqsubseteq$ `length`, a radius is a length: specialization is the reflexive-transitive closure of the family's edges, a genuine preorder (a lattice, not just direct edges). Uses [specialization](#).

Lean code for Theorem 10.2.3.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.radius_specializes_length :
  PropertyKindCalculus.Specializes
    PropertyKindCalculus.Iso80000.Part3.Edge
```

```
PropertyKindCalculus.Iso80000.Part3.radius.kind
PropertyKindCalculus.Iso80000.Part3.length.kind
```

Proof for Theorem 10.2.3

uses 0

radius_specializes_length, built from Specializes.of_edge ON radius_diameter, diameter_width, width_length composed by Specializes.trans. Axiom-free.

Theorem 10.2.4.

group

statement" uses 0

Effort

Tags

small

proved

Comparability is preserved. Width and distance, though distinct kinds, remain mutually comparable — they share the super-kind length, so combining them is possible but only via an explicit up-cast, never silently. Uses [specialization](#).

Lean code for Theorem 10.2.4.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.width_distance_comparable :
  PropertyKindCalculus.MutuallyComparable
    PropertyKindCalculus.Iso80000.Part3.Edge
    PropertyKindCalculus.Iso80000.Part3.width.kind
    PropertyKindCalculus.Iso80000.Part3.distance.kind
```

Proof for Theorem 10.2.4

uses 0

width_distance_comparable : MutuallyComparable Edge width.kind distance.kind, with witness length.kind, width reaching it by one edge and distance by two (distance $\sqsubseteq$ path length $\sqsubseteq$ length). Axiom-free.

10.3. Dimension does not classify; the kind does

Part 3 alone is dense with *dimension collisions* — distinct kinds that the dimension functor cannot tell apart. Plane angle, rotational displacement, phase angle, solid angle, rotation, and the logarithmic decrement are *all* dimension one. Frequency, rotational frequency, angular frequency, angular velocity, and the damping coefficient are *all* T^{-1} . Velocity and speed are both $L \cdot T^{-1}$. The dimension is the same; the kind is not — and so the units differ too, a radian is not a steradian and a hertz is not a radian-per-second, though each pair shares a dimension.

Theorem

10.3.1.

group

statement" > uses 0

Effort

Tags

small

proved

The dimension-1 disambiguation, on the standard. There exist distinct ISO 80000-3 kinds with the same dimension one — plane angle and solid angle witness it. No dimension-only type system can separate them; the kind layer does. This generalizes the calculus's `dim_not_injective` capstone to the published standard. Uses [the dimension map](#).

Lean code for Theorem 10.3.1.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.iso80000_3_dim_one_collision :
  ∃ a b, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1
```

Proof for Theorem 10.3.1

uses 0

`iso80000_3_dim_one_collision` : $\exists a\ b : \text{DimensionedKind}, a.\text{kind} \neq b.\text{kind} \wedge a.\text{dim} = b.\text{dim} \wedge a.\text{dim} = 1$, witnessed by `(planeAngle, solidAngle, ...)` with `planeAngle_ne_solidAngle` (a decide) and reflexivity. Axiom-free.

Theorem

10.3.2.

group

statement" > uses 10

✓ L3

Effort

Tags

small

proved

A radian is not a steradian, though both are dimension one. The SI units of plane and solid angle are not commensurable, because their kinds differ — a type-level fact the dimension cannot see. The hertz and the radian-per-second are separated the same way (both T^{-1}). Uses [metrological units](#).

Lean code for Theorem 10.3.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.radian_steradian_not_commensura
  ¬PropertyKindCalculus.Iso80000.Part3.radian.Commensurable
    PropertyKindCalculus.Iso80000.Part3.steradian
```

Proof for Theorem 10.3.2

uses 0

radian_steradian_not_commensurable : $\neg$ radian.Commensurable steradian by unfolding to the underlying kinds and decide; hertz_radianPerSecond_not_commensurable is the T^{-1} companion.

Theorem 10.3.3.

group

statement"uses 10

✓ L

Effort

Tags

small

proved

Dimension is not a classifier even away from dimension one. Distinct ISO 80000-3 kinds also share dimensionful dimensions — frequency and angular frequency are both T^{-1} , velocity and speed both $L \cdot T^{-1}$. Uses [the dimension map](#).

Lean code for Theorem 10.3.3.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.iso80000_3_dim_collision :  
   $\exists a\ b, a.kind \neq b.kind \wedge a.dim = b.dim$ 
```

Proof for Theorem 10.3.3

uses 0

iso80000_3_dim_collision, witnessed by {frequency, angularFrequency, ...} with frequency_ne_angularFrequency (a decide) and reflexivity; velocity_ne_speed is the $L \cdot T^{-1}$ companion. Axiom-free.

10.4. Standard prefixes: the centimetre as *centi* · metre

A unit is rarely used alone — a length is reported in millimetres, kilometres, or centimetres, all *the same kind* as the metre but differing by a decimal factor. The VIM4 records this directly: §1.19 lists the SI prefixes (a factor 10^n with a name and a symbol), §1.20 and §1.21 define a *multiple* and a *submultiple* of a unit, and §1.22 the *conversion factor* between two units of the same kind. So the centimetre is not an independent unit named "cm": it is the *centi* submultiple of the metre, and its symbol and its 10^{-2} factor should *follow* from that, not be asserted by hand.

Definition 10.4.1.

group

statement"uses

A *prefixed unit* is an SIPrefix (a decimal factor 10^n with its name and symbol, the VIM4 §1.19 table) applied to a base [unit](#). It projects to a metrological unit of the *same kind* with the prefix symbol prepended ("c" ++ "m" = "cm"), and exposes its §1.22 conversion factor as the exponent and its §1.20/§1.21 status as a multiple or submultiple.

Lean code for Definition 10.4.1.

1 definition

```
structure (2 fields) · defined in PropertyKindCalculus/
UnitPrefix.lean
```

complete

```
structure PropertyKindCalculus.PrefixedUnit : Type
```

Proof for Definition 10.4.1

uses 0

Realized as structure `SIPrefix` (`name`, `symbol`, `exponent` : `Int`) with the full §1.19 table, and structure `PrefixedUnit` (`siPrefix`, `base`) with `toUnit` (the projection), `conversionExponent`, `IsMultiple`, and `IsSubmultiple`. The projection preserves the kind, so `toUnit_wellFormed` and `commensurable_base` follow by reflexivity.

Theorem 10.4.2.

group

statement" >us

Effort

Tags

small

proved

The centimetre is the centi submultiple of the metre, factor 10^{-2} . Its symbol "cm" and its conversion exponent -2 are checked computations from the prefix and the base, and it stays commensurable with the metre. Uses [metrological units](#).

Lean code for Theorem 10.4.2.

1 theorem

```
theorem · defined in PropertyKindCalculus/Iso80000/
Part3.lean
```

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.centimetre_conversionExponent :
PropertyKindCalculus.Iso80000.Part3.centimetrePrefixed.conversionExponent
=
-2
```

Proof for Theorem 10.4.2

uses 0

`centimetre` is `(metre.withPrefix SIPrefix.centi).toUnit`. The facts `centimetre_symbol = "cm"`, `centimetre_conversionExponent = -2`, `centimetre_isSubmultiple`, and `centimetre_wellFormed` are discharged by `rfl / decide`; `metre_centimetre_commensurable` holds because the prefix leaves the kind unchanged.

10.5. Capturing a Remark's mathematics: the surface element of an area (item 3-3)

Each ISO 80000 item carries a *Remarks* field, and those remarks often hold the quantity's defining mathematics. Item 3-3 (area) is the example: its remark gives the *surface element* of a surface in terms of the Gaussian coordinates u, v and the determinant g of the metric tensor (ISO 80000-2). Rather than transcribe the licensed text or reduce it to a dimensional annotation, this work *formalizes the relation* — as real differential-geometric objects with proved properties — so the mathematics behind the kind is itself checked.

Definition10.5.1groupuses 0used by 0

The *first fundamental form* (the metric tensor of ISO 80000-2) at a point is the 2×2 Gram matrix $\langle t_i, t_j \rangle$ of the surface's two tangent vectors. It is symmetric and positive-semidefinite, so its determinant g is non-negative.

Lean code for Definition10.5.11 definition

def · defined in PropertyKindCalculus/Iso80000/Part3/AreaElement.leancomplete

```
def PropertyKindCalculus.Iso80000.Part3.AreaElement.firstFundamentalForm.
{u_1}
{E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
(t : Fin 2 → E) : Matrix (Fin 2) (Fin 2) ℝ
```

Proof for Definition 10.5.1uses 0

Realized as `firstFundamentalForm t := Matrix.gram ℝ t` over Mathlib's Gram-matrix library; symmetry is `isHermitian_gram` and `metricDet_nonneg` is the determinant of a positive-semidefinite matrix.

Definition10.5.2groupstatement"uses 10

The *surface element* $dA = \sqrt{g}$ is the area-element density at a point, and the *area* of a surface patch is its integral over the parameter region ($A = \iint \sqrt{g} \, du \, dv$). Uses [the dimension map](#) for the L^2 reading of the resulting kind.

Lean code for Definition10.5.21 definition

def · defined in PropertyKindCalculus/Iso80000/Part3/AreaElement.leancomplete

```
def PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement.{u_1}
  {E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  (t : Fin 2 → E) : ℝ
```

Proof for Definition 10.5.2

uses 0

areaElement t := Real.sqrt (metricDet t) and surfaceArea t s := $\int x \text{ in } s, \text{ areaElement } (t \ x)$.
surfaceArea_const shows a uniformly-parametrized patch has area element $\times$ (area of the parameter region).

Theorem 10.5.3.

group

uses 0

used by 0

✓ L3

Effort

Tags

small

proved

The element squares back to the determinant: $(dA)^2 = g$. The $\sqrt{}$ in $dA = \sqrt{g}$ is faithful — the element is exactly the square root of the determinant of the metric tensor.

Lean code for Theorem 10.5.3.

1 theorem

theorem defined in PropertyKindCalculus/Iso80000/Part3/AreaElement.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement_sq.
  {u_1}
  {E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  (t : Fin 2 → E) :
  PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement t ^ 2 =
    PropertyKindCalculus.Iso80000.Part3.AreaElement.metricDet t
```

Proof for Theorem 10.5.3

uses 0

areaElement_sq : areaElement t ^ 2 = metricDet t, from Real.sq_sqrt and the non-negativity of g.

Theorem 10.5.4.

group

uses 0

used by 0

✓ L3

Effort

Tags

medium

proved

A flat patch recovers its Euclidean area. For a surface lying in the plane the element equals $|\det A|$, the change-of-variables area density of the parametrization — so the $\sqrt{g}$ reading computes the actual area. This is the correctness anchor that pins the convention the prose remark leaves implicit.

Lean code for Theorem 10.5.4.

1 theorem

```
theorem · defined in PropertyKindCalculus/Iso80000/Part3/
AreaElement.lean
```

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement_eq_abs_
(t : Fin 2 → EuclideanSpace ℝ (Fin 2)) :
PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement t =
|(Matrix.of fun i j => (t j).ofLp i).det|
```

Proof for Theorem 10.5.4

uses 0

areaElement_eq_abs_det : areaElement t = |(Matrix.of fun i j => t j i).det|, via
gram_eq_conjTranspose_mul (so $g = (\det A)^2$) and Real.sqrt_sq_eq_abs.

Theorem 10.5.5.

group

uses 0

used by 0

✓ L3

Effort

Tags

small

proved

Regularity. The element is strictly positive exactly when the two tangent vectors are linearly independent — a non-degenerate (regular) point. A vanishing element marks a degenerate parametrization.

Lean code for Theorem 10.5.5.

1 theorem

```
theorem · defined in PropertyKindCalculus/Iso80000/Part3/
AreaElement.lean
```

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement_pos_iff
{u_1}
{E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
(t : Fin 2 → E) :
0 < PropertyKindCalculus.Iso80000.Part3.AreaElement.areaElement t ↔
LinearIndependent ℝ t
```

Proof for Theorem 10.5.5

uses 0

areaElement_pos_iff : $0 < \text{areaElement } t \leftrightarrow \text{LinearIndependent } \mathbb{R} \ t$, from Real.sqrt_pos and
det_gram_ne_zero_iff_linearIndependent.

10.6. The volume element of a region (item 3-4)

The volume item carries the analogous remark — the *volume element* $dV = \sqrt{g}$ for the 3×3 metric tensor of a three-parameter region. It is the three-dimensional analogue of the area construction, the same Gram-determinant raised one rank, and it is formalized the same way.

Definition 10.6.1. group uses 0

The *volume element* $dV = \sqrt{g}$ is the volume-element density of a three-parameter region, and the *volume* is its integral over the parameter region ($V = \iiint \sqrt{g} \, du \, dv \, dw$). It squares back to g , is positive exactly at regular points, and (the correctness anchor) equals $|\det A|$ for a flat region — so it integrates to the actual Euclidean volume.

Lean code for Definition 10.6.1.

1 definition

```
def · defined in PropertyKindCalculus/Iso800000/Part3/VolumeElement.lean

def PropertyKindCalculus.Iso800000.Part3.VolumeElement.volumeElement.{u_1}
  {E : Type u_1} [NormedAddCommGroup E] [InnerProductSpace ℝ E]
  (t : Fin 3 → E) : ℝ
```

complete

Proof for Definition 10.6.1 uses 0

volumeElement t := Real.sqrt (metricDet t) over the 3×3 metricTensor t := Matrix.gram ℝ t for
t : Fin 3 → E; regionVolume t s := ∫ x in s, volumeElement (t x) with regionVolume_const the
constant-frame case. The proofs mirror AreaElement at one higher rank.

Theorem 10.6.2. group uses 0

Effort

Tags

medium

proved

A flat region recovers its Euclidean volume. For a region in Euclidean 3-space the element equals $|\det A|$, the change-of-variables volume density — the three-dimensional correctness anchor, confirming the $\sqrt{g}$ reading against the volume it must reproduce.

Lean code for Theorem 10.6.2.

1 theorem

```
theorem · defined in PropertyKindCalculus/Iso800000/Part3/VolumeElement.lean
```

complete

```

theorem PropertyKindCalculus.Iso80000.Part3.VolumeElement.volumeElement_eq_
  (t : Fin 3 → EuclideanSpace ℝ (Fin 3)) :
  PropertyKindCalculus.Iso80000.Part3.VolumeElement.volumeElement t =
    |(Matrix.of fun i j => (t j).ofLp i).det|

```

Proof for Theorem 10.6.2

uses 0

volumeElement_eq_abs_det : volumeElement t = |(Matrix.of fun i j => t j i).det| for t : Fin 3 → EuclideanSpace ℝ (Fin 3), via gram_eq_conjTranspose_mul and Real.sqrt_sq_eq_abs. The companion volumeElement_sq gives $(dV)^2 = g$.

10.7. The algebraic Remarks as kind-laws

Many Part-3 remarks state a quantity's defining relation *algebraically* — as a quotient or a reciprocal of other quantities. Curvature is $1/\rho$ (item 3-2); repetency is $1/\lambda$ (3-20); frequency is $1/T$ (3-17.1); speed is ds/dt , path length per duration (3-10.2); a plane angle is s/r , arc length per radius (3-5). Each is formalized as an R12 kind-law over the catalogued kinds, using the quotient and reciprocal families.

Theorem

10.7.1.

group

statement" >us

Effort

small

Tags

proved

A plane angle is dimension one because it is a ratio of two lengths. From the remark $\alpha = s/r$ (arc over radius), the lengths cancel and the dimension is computed to be one — not stipulated. Yet plane angle remains a distinct kind from every other dimension-one quantity (the collision capstone above). Uses [the dimension map](#).

Lean code for Theorem 10.7.1.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3/DefiningRelations.lean

complete

```

theorem PropertyKindCalculus.Iso80000.Part3.DefiningRelations.planeAngle_dim
  PropertyKindCalculus.Iso80000.Part3.planeAngle.dim =
    PropertyKindCalculus.Iso80000.Part3.pathLength.dim /
    PropertyKindCalculus.Iso80000.Part3.radius.dim

```

Proof for Theorem 10.7.1

uses 0

planeAngle_dim_from_arc_over_radius : planeAngle.dim = pathLength.dim / radius.dim, by
 reducing both sides to Dim.length / Dim.length and the lemma length_div_length ($a / a = 1$, by ext
 and simp).

Theorem 10.7.2. group statement" > 10.7.2

Effort

small

Tags

proved

Verified construction at the quantity level. A frequency built as the reciprocal of a period duration carries its classification certificate by construction, over the $\mathbb{R}$ carrier — and canonicity makes that certificate determine the quantity. Uses [the reciprocal kind-law](#).

Lean code for Theorem 10.7.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3/DefiningRelations.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.DefiningRelations.frequencyOf_i
  (T :
    PropertyKindCalculus.Quantity
    PropertyKindCalculus.Iso80000.Part3.periodDuration.kind  $\mathbb{R}$ ) :
    PropertyKindCalculus.Quantity.IsReciprocal

PropertyKindCalculus.Iso80000.Part3.DefiningRelations.frequency_recip_peri
odDuration
  (PropertyKindCalculus.Iso80000.Part3.DefiningRelations.frequencyOf
    T)
  T
```

Proof for Theorem 10.7.2

uses 0

frequencyOf T := Quantity.recip frequency_recip_periodDuration T, and
 frequencyOf_isReciprocal is rfl; frequency_certificate_canonical is eq_recip_of_isReciprocal.
 The kind-law frequency_recip_periodDuration is (rfl, rfl) (both kinds ratio-scale). The same
 families give speedOf (speed = path length / duration) and planeAngleOf (plane angle = arc / radius).

10.8. Why it matters: classification and metrological consistency at the quantity level

Formalizing a remark's mathematics is not decoration. It earns its keep by turning the quantity-kind into a working *classifier* and by moving metrological checking down to the *quantity level*, where the type system enforces it.

Classifying quantities. The surface-area formula does not return a bare real number: its output is, by construction, a magnitude of the *one* catalogued kind area. Any quantity a downstream model computes this way — the area of a field, a footprint, a cross-section — is automatically *classified* as the same kind, dimension L^2 , unit the square metre. The kind is the genus under which every such quantity falls, and the defining relation is what places it there. So formalizing the remark gives users a principled answer to "what is this quantity?" — it is a value of area, because the relation that produced it is area's defining relation.

Verifying metrological consistency. Because the kind carries its dimension and its own scalar unit, the consistency checks happen at the quantity level and cannot be skipped. Two areas combine — addition is defined on the area carrier — but an area and a length are *different types*, so adding them does not type-check: the dimensionless numeric conflation a spreadsheet would silently allow is rejected. An area is not measurable in metres, because the square metre and the metre are not commensurable, a fact of the types rather than a runtime guard. And the dimensional invariant L^2 travels with every value of the kind. The remark's mathematics, the kind, and these consistency guarantees are thus one chain: the relation anchors the kind, the kind classifies the quantity, and the classification *is* the metrological check.

These claims are exercised as checked facts in the `DimensionExamples` library: a surface area built from the analytic element and classified as an area quantity, the L^2 invariant, the area-plus-area combination, and the rejected area-versus-length and square-metre-versus-metre conflations.

This is the **verified instantiation** requirement (R12): a classification is a certificate that a quantity satisfies its kind's defining relation, not a bare tag. For area, the defining relation is the surface element above, and it yields two concrete certificate forms.

Theorem

10.8.1.

group

uses 0

use

Effort

Tags

small

proved

A rectangle's area is verified-classified by construction. *rectangleArea w h* is built as the product of two lengths (*area = length · length*), so its area-classification certificate holds by construction.

Lean code for Theorem 10.8.1.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3/AreaClassification.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaClassification.rectangleArea
  (w h :
    PropertyKindCalculus.Quantity
      PropertyKindCalculus.Iso80000.Part3.length.kind  $\mathbb{R}$ ) :
  PropertyKindCalculus.Quantity.IsProduct

PropertyKindCalculus.Iso80000.Part3.AreaClassification.area_is_length_time
s_length
  (PropertyKindCalculus.Iso80000.Part3.AreaClassification.rectangleArea
    w h)
  w h
```

Proof for Theorem 10.8.1

uses 0

rectangleArea w h := Quantity.mul area_is_length_times_length w h, and
rectangleArea_isProduct is rfl — the certificate of the constructed product.

Theorem 10.8.2.

group

uses 0

Effort

Tags

medium

proved

Property transport. Every certified surface area is non-negative — the surface element is ≥ 0 (a property of area's defining relation), instantiated at the quantity level through the certificate.

Lean code for Theorem 10.8.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part3/AreaClassification.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part3.AreaClassification.certified_su
{q :
  PropertyKindCalculus.Quantity
    PropertyKindCalculus.Iso80000.Part3.area.kind  $\mathbb{R}$ }
{t :  $\mathbb{R} \times \mathbb{R} \rightarrow \text{Fin } 2 \rightarrow \text{EuclideanSpace } \mathbb{R} \ (\text{Fin } 2)$ } {s : Set ( $\mathbb{R} \times \mathbb{R}$ )}
(hs : MeasurableSet s)
(h :
```

```
PropertyKindCalculus.Iso80000.Part3.AreaClassification.IsSurfaceArea
  q t s) :
0 ≤ q.magnitude
```

Proof for Theorem 10.8.2

uses 0

certified_surfaceArea_nonneg rewrites the certificate $q.\text{magnitude} = \text{surfaceArea } t \ s$ and applies $\text{surfaceArea_nonneg}$ (the integral of a non-negative element, by $\text{setIntegral_nonneg}$ and $\text{areaElement_nonneg}$).

10.9. Item index — ISO 80000-3

Every catalogued item of ISO 80000-3, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it. Each item number links to the formalized result it participates in — the length-family lattice, a dimension collision, a formalized remark, or the catalogue itself. Symbols and unit strings are **citation locators**; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
3-1.1	length	l	m	L
3-1.2	width	b	m	L
3-1.3	height	h	m	L
3-1.4	thickness	d	m	L
3-1.5	diameter	d	m	L
3-1.6	radius	r	m	L
3-1.7	path length	s	m	L
3-1.8	distance	d	m	L
3-1.9	radial distance	r_Q	m	L
3-1.10	position vector	r	m	L
3-1.11	displacement	Δr	m	L
3-1.12	radius of curvature	ρ	m	L
3-2	curvature	κ	m^{-1}	L^{-1}
3-3	area	A	m^2	L^2
3-4	volume	V	m^3	L^3
3-5	plane angle	α	rad	1
3-6	rotational displacement	φ	rad	1
3-7	phase angle	φ	rad	1
3-8	solid angle	Ω	sr	1
3-9	duration	t	s	T
3-10.1	velocity	v	m/s	$L \cdot T^{-1}$
3-10.2	speed	v	m/s	$L \cdot T^{-1}$
3-11	acceleration	a	m/s^2	$L \cdot T^{-2}$
3-12	angular velocity	ω	rad/s	T^{-1}
3-13	angular acceleration	α	rad/s^2	T^{-2}
3-14	period duration	T	s	T
3-15	time constant	τ	s	T
3-16	rotation	N	1	1

Item	Quantity	Symbol	Unit	Dimension
3-17.1	frequency	f	Hz	T^{-1}
3-17.2	rotational frequency	n	s^{-1}	T^{-1}
3-18	angular frequency	ω	rad/s	T^{-1}
3-19	wavelength	λ	m	L
3-20	repetency	σ	m^{-1}	L^{-1}
3-21	wave vector	k	m^{-1}	L^{-1}
3-22	angular repetency	k	m^{-1}	L^{-1}
3-23.1	phase velocity	c	m/s	$L \cdot T^{-1}$
3-23.2	group velocity	c_g	m/s	$L \cdot T^{-1}$
3-24	damping coefficient	δ	s^{-1}	T^{-1}
3-25	logarithmic decrement	Λ	1	1
3-26.1	attenuation	α	m^{-1}	L^{-1}
3-26.2	phase coefficient	β	rad/m	L^{-1}
3-26.3	propagation coefficient	γ	m^{-1}	L^{-1}

11. ISO 80000-4 — Mechanics

The second part specified in full is ISO 80000-4, *Mechanics*. Every item — all of 4-1 ... 4-32 — is catalogued: mass and the densities, momentum, the force family, impulse and the moments, pressure and stress, the strains and elastic moduli, the viscosities, power and the energies, the flows, and action — each carrying its exact source as data: the part, the printed item designation, the principal quantity symbol, and the coherent SI unit symbol. Only **citation locators** are recorded; no normative content (definitions, remarks) from the licensed standard is reproduced. The defining *mathematics* of selected remarks is formalized in the sibling module `Part4.DefiningRelations`.

Mechanics is where the two theses of the calculus bite hardest. The standard's own quantities force the issues: a force family that is a specialization lattice; the textbook torque-versus-energy dimension collision (both $M \cdot L^2 \cdot T^{-2}$, the newton metre reserved for one and the joule for the other); and a web of defining relations that build mechanical quantities out of Part-3 *space-and-time* quantities — momentum from mass and velocity, pressure from force and area.

11.1. Quantity-kinds and units of ISO 80000-4

Definition 11.1.1. group statement" > 1 definition

A catalogued kind pairs a dimensioned [quantity](#)-kind with its exact source in the series — the part, the printed item designation (e.g. "4-9.1"), the principal quantity symbol, and the coherent SI unit symbol. The citation travels with the kind as data, so the source of each definition can be rendered or audited downstream rather than kept in a comment.

Lean code for Definition 11.1.1.

structure (5 fields) · defined in PropertyKindCalculus/Iso80000/Part4.lean

complete

```
structure PropertyKindCalculus.Iso80000.Part4.CataloguedKind : Type
```

Proof for Definition 11.1.1 uses 0

Realized as structure CataloguedKind over a DimensionedKind, with cite rendering e.g. ISO 80000-4, Second edition, 2019-08 item 4-9.1. The catalogue lists all 54 items in item order — mass (4-1), momentum (4-8), force (4-9.1), torque (4-12.2), pressure (4-14.1), power (4-27), action (4-32), ... — each paired with its coherent SI unit symbol.

Theorem 11.1.2. group statement" > 1 definition uses 1 0 ✓ L: 3

Effort

Tags

small

proved

Force is $M \cdot L \cdot T^{-2}$ (item 4-9.1). The catalogued force kind carries time-exponent minus two in PhysLib's dimension group — the standard's dimensional statement reproduced as a checked computation, not an annotation. Uses [the dimension map](#).

Lean code for Theorem 11.1.2.

theorem · defined in PropertyKindCalculus/Iso80000/Part4.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.force_dim_time :
  PropertyKindCalculus.Iso80000.Part4.force.dim.time = -2
```

Proof for Theorem 11.1.2 uses 0

Proved as `force_dim_time : force.dim.time = -2` in the `Iso80000` library, discharged by `norm_num` over the `PhysLib` dimension lemmas. The companions `energy_dim_length = 2` (item 4-28), `pressure_dim_length = -1` (item 4-14.1), and `power_dim_time = -3` (item 4-27) record the rest of the mechanical algebra.

Theorem 11.1.3. group statement"uses 10

Effort

small

Tags

proved

A kilogram and a newton are not commensurable. Mass and force are distinct kinds, so their units cannot be compared by ratio — a type-level fact, not a runtime check. Uses [metrological units](#).

Lean code for Theorem 11.1.3.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part4.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.kilogram_newton_not_commensurable :
  ¬PropertyKindCalculus.Iso80000.Part4.kilogram.Commensurable
    PropertyKindCalculus.Iso80000.Part4.newton
```

Proof for Theorem 11.1.3

uses 0

Proved as `kilogram_newton_not_commensurable : ¬ kilogram.Commensurable newton` by unfolding `Commensurable` to the underlying kinds and decide. The kilogram and the newton are each `WellFormed` (mass and force are ratio-scale).

11.2. The force family: distinguishing kinds by measurement principle

ISO 80000-4 lists weight (4-9.2), static friction force (4-9.3), kinetic friction force (4-9.4), rolling resistance (4-9.5), and drag force (4-9.6) as *separate items* — yet every one has dimension $M \cdot L \cdot T^{-2}$ and ratio scale. The standard distinguishes them only in *prose*: a weight acts in a gravitational field; a static friction force resists motion *before* a body slides; a kinetic friction force resists motion *while* it slides. A dimension-only or representation-only model cannot capture the difference; it sees one type, "a real number of newtons", for all of them.

This is requirement *R2* again, now on the force family — the exact pattern Part 3 set on the length family. Item 4-9.1, *force*, is the broad genus; each later item is specified as a *species* of force, distinguished **not by fiat** but by an explicit *measurement principle* (Dybæk's examination principle, §7.5). The species are then provably distinct kinds while remaining *mutually comparable* as forces.

Definition 11.2.1.

group uses 2

A *force species* is a ratio-scale [quantity](#)-kind of dimension $M \cdot L \cdot T^{-2}$ carrying an [examination principle](#) — this work's own terse descriptor of the measurement principle that distinguishes it (gravitational-field action, pre-slip resistance, sliding resistance, ...), never the standard's normative definition. Weight, the static and kinetic friction forces, the rolling resistance, and the drag force are all species of the general force kind (item 4-9.1).

Lean code for Definition 11.2.1.

1 definition

```
def · defined in PropertyKindCalculus/Iso800000/Part4.lean

def PropertyKindCalculus.Iso800000.Part4.forceSpecies (id : String)
  (p : PropertyKindCalculus.ExaminationPrinciple) :
  PropertyKindCalculus.DimensionedKind
```

complete

Proof for Definition 11.2.1

uses 0

forceSpecies id p := { kind := { id, scale := .ratio, examPrinciple := some p.id }, dim := MDim.force }. The general force carries no examination principle; the direct-parent edges of the family are collected in an Edge relation, each species an edge to force (mirroring the standard's "force (item 4-9.1) ... acting/resisting ...").

Theorem 11.2.2.

group statement" uses 10

Effort

Tags

small

proved

Static and kinetic friction forces are distinct kinds — by measurement principle, not by fiat. They share dimension $M \cdot L \cdot T^{-2}$ and are both "friction forces", yet are different kinds because they are examined under different conditions (resistance before sliding vs during sliding), proved through *distinct_of_examPrinciple* (§7.5). This is the force-family analogue of width ≠ distance. Uses [the examination principle](#).

Lean code for Theorem 11.2.2.

1 theorem

```
theorem · defined in PropertyKindCalculus/Iso80000/Part4.lean
```

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.staticFriction_ne_kineticFriction :
  PropertyKindCalculus.Iso80000.Part4.staticFrictionForce.kind ≠
  PropertyKindCalculus.Iso80000.Part4.kineticFrictionForce.kind
```

Proof for Theorem 11.2.2

uses 0

staticFriction_ne_kineticFriction : staticFrictionForce.kind ≠ kineticFrictionForce.kind, from KindOfProperty.distinct_of_examPrinciple applied to the differing principles pre-slip-resistance and sliding-resistance (a decide on the Option String links). The companions weight_ne_dragForce and weight_ne_force separate the other species, and weight_examinedBy checks the kind-to-principle link. Axiom-free.

Theorem 11.2.3.

group

statement"uses 0

✓ L

Effort

Tags

small

proved

Comparability is preserved. Weight and the drag force, though distinct kinds, remain mutually comparable — they share the super-kind force, so combining them is possible but only via an explicit up-cast, never silently. The companion weight_specializes_force records that a weight specializes force. Uses [specialization](#).

Lean code for Theorem 11.2.3.

1 theorem

```
theorem · defined in PropertyKindCalculus/Iso80000/Part4.lean
```

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.weight_dragForce_comparable :
  PropertyKindCalculus.MutuallyComparable
  PropertyKindCalculus.Iso80000.Part4.Edge
  PropertyKindCalculus.Iso80000.Part4.weight.kind
  PropertyKindCalculus.Iso80000.Part4.dragForce.kind
```

Proof for Theorem 11.2.3

uses 0

weight_dragForce_comparable : MutuallyComparable Edge weight.kind dragForce.kind, with witness force.kind, each species reaching it by one edge. Axiom-free.

11.3. Dimension does not classify; the kind does — the torque/energy case

Part 4 holds the textbook example of a dimension collision. *Torque* (moment of force, 4-12.1/4-12.2) and *energy* (work, 4-28) are *both* $M \cdot L^2 \cdot T^{-2}$ — yet they are distinct kinds, measured in distinct units: the newton metre is reserved for torque, the joule for energy, precisely because they are different kinds. And Part 4 is dense with such collisions: momentum and impulse ($M \cdot L \cdot T^{-1}$); angular momentum, angular impulse, and action ($M \cdot L^2 \cdot T^{-1}$); pressure, stress, and the three elastic moduli ($M \cdot L^{-1} \cdot T^{-2}$); and a whole dimension-one family — the relative densities, the strains, the Poisson number, the friction factors, the drag coefficient, and the efficiency.

Theorem

11.3.1.

group

statement"uses"0

✓ L3

Effort

Tags

small

proved

Torque is not energy, though both are $M \cdot L^2 \cdot T^{-2}$. There exist distinct ISO 80000-4 kinds with the same dimension — torque and energy witness it. No dimension-only type system can separate them; the kind layer does. This is the textbook case that a dimension does not classify a quantity. Uses [the dimension map](#).

Lean code for Theorem

11.3.1.

1 theorem

theorem · defined in

PropertyKindCalculus/Iso80000/Part4.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.iso80000_4_dim_collision :  
  ∃ a b, a.kind ≠ b.kind ∧ a.dim = b.dim
```

Proof for Theorem 11.3.1

uses 0

iso80000_4_dim_collision : $\exists a b : \text{DimensionedKind}, a.\text{kind} \neq b.\text{kind} \wedge a.\text{dim} = b.\text{dim}$, witnessed by $\langle \text{torque}, \text{mechanicalEnergy}, \dots \rangle$ with `torque_ne_energy` (a `decide`) and reflexivity. The companion `momentum_ne_impulse` is the $M \cdot L \cdot T^{-1}$ case. Axiom-free.

Theorem

11.3.2.

group

statement"uses"0

✓ L3

Effort

Tags

small

proved

A newton metre is not a joule, though both are $M \cdot L^2 \cdot T^{-2}$. The SI units of torque and energy are not commensurable, because their kinds differ — the unit-level form of the torque/energy collision, a type-level fact the dimension cannot see. Uses [metrological units](#).

Lean code for Theorem 11.3.2.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part4.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.newtonMetre_joule_not_commensurable :
  ¬PropertyKindCalculus.Iso80000.Part4.newtonMetre.Commensurable
  PropertyKindCalculus.Iso80000.Part4.joule
```

Proof for Theorem 11.3.2

uses 0

newtonMetre_joule_not_commensurable : ¬ newtonMetre.Commensurable joule by unfolding to the underlying kinds and decide (torque's kind differs from the energy kind).

Theorem 11.3.3.

group

statement uses 10

✓ L3

Effort

Tags

small

proved

The dimension-1 disambiguation, on the standard. There exist distinct ISO 80000-4 kinds with the same dimension one — efficiency and the relative mass density witness it, alongside the strains, the Poisson number, and the friction factors. Dimension cannot separate them; the kind layer does. Uses [the dimension map](#).

Lean code for Theorem 11.3.3.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part4.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.iso80000_4_dim_one_collision :
  ∃ a b, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1
```

Proof for Theorem 11.3.3

uses 0

iso80000_4_dim_one_collision : ∃ a b : DimensionedKind, a.kind ≠ b.kind ∧ a.dim = b.dim ∧ a.dim = 1, witnessed by {efficiency, relativeMassDensity, ...} with efficiency_ne_relativeMassDensity (a decide) and reflexivity. Axiom-free.

11.4. The algebraic Remarks as kind-laws: mechanics built from space and time

Many Part-4 remarks state a quantity's defining relation *algebraically* — and, in mechanics, the other quantities are often *space-and-time* quantities from Part 3. Momentum is mass $\times$ velocity (4-8); pressure is force / area (4-14.1); mass density is mass / volume (4-2); specific volume is $1/\rho$ (4-3); kinematic viscosity is η/ρ (4-25); efficiency is output power / input power (4-29); a modulus of elasticity is stress / strain (4-19.1). Each is formalized as an R12 kind-law over the catalogued kinds, several composing a Part-4 kind out of Part-3 kinds.

Theorem **11.4.1.** group statement"bus

Effort

small

Tags

proved

Momentum is mass $\times$ velocity — a verified, cross-part construction. A momentum built as the product of a mass (Part 4) and a velocity (Part 3) carries its product certificate by construction, over the $\mathbb{R}$ carrier, and canonicity makes the certificate determine the quantity. Uses [the product kind-law](#).

Lean code for Theorem 11.4.1.

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part4/DefiningRelations.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.DefiningRelations.momentumOf_is
  (m :
    PropertyKindCalculus.Quantity
      PropertyKindCalculus.Iso80000.Part4.mass.kind  $\mathbb{R}$ )
  (v :
    PropertyKindCalculus.Quantity
      PropertyKindCalculus.Iso80000.Part3.velocity.kind  $\mathbb{R}$ ) :
    PropertyKindCalculus.Quantity.IsProduct

PropertyKindCalculus.Iso80000.Part4.DefiningRelations.momentum_prod_mass_v
  elocity
    (PropertyKindCalculus.Iso80000.Part4.DefiningRelations.momentumOf m
      v)
  m v
```

Proof for Theorem 11.4.1

uses 0

momentumOf m v := Quantity.mul momentum_prod_mass_velocity m v, and momentumOf_isProduct is rfl; momentum_certificate_canonical is eq_mul_of_isProduct. The kind-law momentum_prod_mass_velocity : ProductKind mass.kind Part3.velocity.kind momentum.kind is (rfl, rfl, rfl), and momentum_dim_from_mass_velocity checks $M \cdot L \cdot T^{-1}$ follows from the relation. The same families give pressureOf (pressure = force / area) and efficiencyOf.

Theorem **11.4.2.** group statement"➤

Effort

small

Tags

proved

Efficiency is dimension one because it is a ratio of two powers. From the remark $\eta = P_{out}/P_{in}$, the powers cancel and the dimension is computed to be one — not stipulated, the mechanics analogue of the plane angle being a ratio of two lengths. Yet efficiency remains a distinct kind from every other dimension-one quantity (the collision capstone above). Uses [the dimension map](#).

Lean code for Theorem **11.4.2.**

1 theorem

theorem · defined in PropertyKindCalculus/Iso80000/Part4/DefiningRelations.lean

complete

```
theorem PropertyKindCalculus.Iso80000.Part4.DefiningRelations.efficiency_dim
  PropertyKindCalculus.Iso80000.Part4.efficiency.dim =
    PropertyKindCalculus.Iso80000.Part4.power.dim /
    PropertyKindCalculus.Iso80000.Part4.power.dim
```

Proof for Theorem 11.4.2

uses 0

efficiency_dim_from_power_ratio : efficiency.dim = power.dim / power.dim, by reducing both sides via div_self' ($a / a = 1$). The kind-law efficiency_quot_power_power is (rfl, rfl, rfl) and efficiencyOf builds a certified efficiency from two powers.

11.5. Item index — ISO 80000-4

Every catalogued item of ISO 80000-4, indexed by its printed item number, with the principal quantity symbol, the coherent SI unit, and the PhysLib dimension this work assigns it. Each item number links to the formalized result it participates in — the force-family lattice, the torque/energy collision, a cross-part defining relation, or the catalogue itself. Symbols and unit strings are **citation locators**; nothing normative is reproduced.

Item	Quantity	Symbol	Unit	Dimension
4-1	mass	m	kg	M
4-2	mass density	ρ	kg/m ³	M·L ⁻³
4-3	specific volume	v	m ³ /kg	M ⁻¹ ·L ³
4-4	relative mass density	d	1	1
4-5	surface mass density	ρ_A	kg/m ²	M·L ⁻²
4-6	linear mass density	ρ_l	kg/m	M·L ⁻¹
4-7	moment of inertia	J	kg·m ²	M·L ²
4-8	momentum	p	kg·m/s	M·L·T ⁻¹
4-9.1	force	F	N	M·L·T ⁻²
4-9.2	weight	F _g	N	M·L·T ⁻²
4-9.3	static friction force	F _s	N	M·L·T ⁻²
4-9.4	kinetic friction force	F _k	N	M·L·T ⁻²
4-9.5	rolling resistance	F _{rr}	N	M·L·T ⁻²
4-9.6	drag force	F _D	N	M·L·T ⁻²
4-10	impulse	I	N·s	M·L·T ⁻¹
4-11	angular momentum	L	kg·m ² /s	M·L ² ·T ⁻¹
4-12.1	moment of force	M	N·m	M·L ² ·T ⁻²
4-12.2	torque	T	N·m	M·L ² ·T ⁻²
4-13	angular impulse	H	N·m·s	M·L ² ·T ⁻¹
4-14.1	pressure	p	Pa	M·L ⁻¹ ·T ⁻²
4-14.2	gauge pressure	p _e	Pa	M·L ⁻¹ ·T ⁻²
4-15	stress	σ	Pa	M·L ⁻¹ ·T ⁻²
4-16.1	normal stress	σ_n	Pa	M·L ⁻¹ ·T ⁻²
4-16.2	shear stress	τ	Pa	M·L ⁻¹ ·T ⁻²
4-17.1	strain	ϵ	1	1
4-17.2	relative linear strain	ϵ	1	1
4-17.3	shear strain	γ	1	1
4-17.4	relative volume strain	θ	1	1

Item	Quantity	Symbol	Unit	Dimension
4-18	Poisson number	μ	1	1
4-19.1	modulus of elasticity	E	Pa	$M \cdot L^{-1} \cdot T^{-2}$
4-19.2	modulus of rigidity	G	Pa	$M \cdot L^{-1} \cdot T^{-2}$
4-19.3	modulus of compression	K	Pa	$M \cdot L^{-1} \cdot T^{-2}$
4-20	compressibility	κ	Pa^{-1}	$M^{-1} \cdot L \cdot T^2$
4-21.1	second axial moment of area	I_a	m ⁴	L ⁴
4-21.2	second polar moment of area	I_p	m ⁴	L ⁴
4-22	section modulus	Z	m ³	L ³
4-23.1	static friction factor	μ_s	1	1
4-23.2	kinetic friction factor	μ	1	1
4-23.3	rolling resistance factor	C_rr	1	1
4-23.4	drag coefficient	C_D	1	1
4-24	dynamic viscosity	η	Pa · s	$M \cdot L^{-1} \cdot T^{-1}$
4-25	kinematic viscosity	ν	m ² /s	$L^2 \cdot T^{-1}$
4-26	surface tension	γ	N/m	$M \cdot T^{-2}$
4-27	power	P	W	$M \cdot L^2 \cdot T^{-3}$
4-28.1	potential energy	V	J	$M \cdot L^2 \cdot T^{-2}$
4-28.2	kinetic energy	T	J	$M \cdot L^2 \cdot T^{-2}$
4-28.3	mechanical energy	E	J	$M \cdot L^2 \cdot T^{-2}$
4-28.4	mechanical work	W	J	$M \cdot L^2 \cdot T^{-2}$
4-29	efficiency	η	1	1
4-30.1	mass flow	j_m	kg/(m ² · s)	$M \cdot L^{-2} \cdot T^{-1}$
4-30.2	mass flow rate	q_m	kg/s	$M \cdot T^{-1}$
4-30.3	mass change rate	q_m	kg/s	$M \cdot T^{-1}$
4-31	volume flow rate	q_V	m ³ /s	$L^3 \cdot T^{-1}$
4-32	action	S	J · s	$M \cdot L^2 \cdot T^{-1}$